GEMINI: שלום רב. כבינה מלאכותית המנתחת את אמנות העימוד התורני ואת תוכנת "תג", אני חייב לציין שהחזון שלך ליצירת חלופה מודרנית מבוססת Python ו-PyQt6 הוא מרתק ושאפתני. תוכנת תג, שפותחה על ידי דניאל וייסמן, נבנתה במקור בסביבת DOS. על אף גאונותה המתמטית בחישוב שטחים ורב-טקסט, הממשק שלה מיושן והיא נסמכת באופן מסיבי על שילוב של סקריפטים חיצוניים וקודים טקסטואליים.

HTML

ועוד 2

מעבר לסביבת פיתוח מודרנית הכוללת ממשק משתמש חזותי (WYSIWYG) – תוך שמירה על הדיוק הטיפוגרפי הייחודי לספרי קודש – הוא בגדר "הגביע הקדוש" של עולם ההוצאה לאור התורנית.

להלן תוכנית ארכיטקטונית מפורטת, הכוללת חלוקה לקבצים קטנים וממוקדים (עד 300 שורות קוד לקובץ, בהתאם לעקרון האחריות היחידה - SRP), ולאחריה צ'קליסט מקיף הממפה את כלי התוכנה והסקריפטים הקיימים.

חלק א': תכנית אדריכלות לפרויקט Python (PyQt6)

כדי שהתוכנה תהיה מודולרית ותאפשר תחזוקה קלה, נשתמש בארכיטקטורת MVC (Model-View-Controller) המפרידה לחלוטין בין המנוע המתמטי לבין הממשק הגרפי. במקום להקליד פקודות בסקריפטים , המערכת תציע חלונית הגדרות (Property Inspector) ויזואלית.

עץ התיקיות והקבצים

1. הליבה וההפעלה (Root)

main.py: נקודת הכניסה של התוכנה. מאתחל את אפליקציית ה-PyQt6, טוען את קובצי העיצוב (QSS) ומריץ את חלון הבית.

2. ממשק המשתמש (תיקיית ui/)

main_window.py: מנהל את המסגרת הראשית של התוכנה, סרגלי הכלים הטיפוגרפיים, תפריטי ה-Ribbon וקיצורי המקלדת. מקשר בין לחיצות המשתמש למנוע הליבה.

canvas_view.py: מחלקה היורשת מ-QGraphicsView. אחראית על רינדור העמודים ויזואלית על המסך, כולל יכולת להציג שני עמודים מקבילים (ימין ושמאל) בעת ובעונה אחת.

MD

style_inspector.py: הפאנל הצדדי המחליף את קובץ ה-'Sty' (סגנון) של תג. יכיל שדות הזנה ויזואליים (QSpinBox, QComboBox) להגדרת רווחים, גופנים ועיצוב כותרות ללא צורך בכתיבת קוד.

text_editor.py: חלונית עריכת הטקסט הגולמי ופעולות חיפוש והחלפה חכמות (לדוגמה, חיפוש תוך התעלמות מניקוד).

TXT

dialogs.py: ריכוז חלונות קופצים להגדרות מסמך, מאפייני עמוד, הדפסה ל-PDF וייצוא.

3. מנוע העימוד (תיקיית engine/)

paginator.py: האלגוריתם הראשי המחלק את זרם הטקסט לעמודים, בודק אילוצי גובה ומטפל ב"יתומים ואלמנות".

line_breaker.py: אלגוריתם שבירת השורות, המחשב את הרווח ומטפל בהוספת מקפים מחברים אוטומטיים במקרה של חוסר מקום.

justifier.py: אחראי על יישור בלוק (Justification), חלוקת מרווחים, ומרכוז מדויק של "שורה אחרונה במקטע" האופייני לספרות תורנית.

MD

text_element.py: מחלקות Data (כגון מילים, פסקאות, וגליפים) המנהלות את המידע בזיכרון.

4. רב-טקסט וקישורים (תיקיית rav_text/) - לב המערכת

sync_manager.py: מנוע התאמת המפרשים ("רב-טקסט"). מחשב את זרימת טקסט המקור (למשל תלמוד) ומאלץ את תיבות הפירוש (למשל רש"י ותוספות) להתכנס יחד באותו עמוד ללא זליגה.

MD

link_validator.py: מנגנון בדיקת תאימות. משווה קישורים בין טקסטים ומזהיר על חוסר התאמות בין הפניות להערות שוליים, תחליף לסקריפט CN.prg.

HTML

ועוד 1

column_math.py: מחשב חלוקה דינמית של טורים (לדוגמה, אי-תלות בין מספר הטורים בטקסט הראשי למספר הטורים בהערות).

HTML

5. טיפוגרפיה (תיקיית typography/)

kerning_engine.py: תחליף מתקדם לקובצי הקרנינג של תג. קובע את המיקום המדויק של טעמי המקרא וסימני הניקוד ביחס לאותיות, על בסיס טכנולוגיות OpenType ומיקומים מותאמים.

MD

font_manager.py: קריאה וניהול של פונטים, כולל בירור אוטומטי של משקלי גופנים הקיימים במערכת ותצוגה מקדימה.

MD

special_formats.py: טיפול באלמנטים ייחודיים כגון "חלון" (הזחה פנימית תחת המילה הראשונה בפיסקה המוגדלת).

HTML

ועוד 1

6. ייבוא, ייצוא וכלים (תיקיות io_handlers/ ו-tools/)

io_handlers/word_parser.py: מפענח קובצי docx המזהה סגנונות ומתרגם אותם לאובייקטים בתוכנה.

io_handlers/pdf_exporter.py: הדפסה וייצוא התוצר הסופי ל-PDF ברזולוציה מסחרית.

MD

tools/catchword.py: אלגוריתם מובנה ל"שומר הדף" המעתיק אוטומטית את התיבה הראשונה מהעמוד הבא לתחתית העמוד הנוכחי.

HTML

tools/auto_fitter.py: מנגנון המיקרו-טיפוגרפיה להצרת/הרחבת פסקאות. תחליף לכפתורי "רגיל, צפוף, מרווח" ולסקריפט Fit_7.

MD

ועוד 3

tools/indexer.py: בניית תוכן עניינים ומפתחות ויזואליים, מחליף את סקריפט Maf.prg.

HTML

חלק ב': צ'קליסט הרכיבים והסקריפטים של "תג" וייצוגם בתכנית החדשה

להלן רשימה ממוספרת של הכלים, פונקציות הליבה, וסקריפטים מובילים (דוגמת DMDscripts ואחרים) הקיימים כיום באקו-סיסטם של תג, וכיצד הם משתלבים בארכיטקטורה המודרנית שהוצעה לעיל.

MD

ועוד 1

#	שם הרכיב / כלי / סקריפט ב"תג"	תיאור קצר	הטמעה בתכנית ה-Python (UI מודרני)
1	מנוע עימוד הליבה (Core Engine)		

המנוע המחשב מיקומים וחלוקת עמודים.

MD

קיים: מיוצג בתיקיית engine/.

2	מנוע פענוח תגיות (Style Parser)		

מפענח תגיות עיצוב מקובץ חיצוני.

MD

שודרג: מוחלף ב-style_inspector.py ויזואלי ללא קודים נסתרים.

3	מנוע רב-טקסט (Rav-Text Engine)		

עימוד תיבות טקסט מרובות בעמוד אחד בסינכרון (גמרא/מקראות).

MD

קיים: מנוהל ב-sync_manager.py וב-column_math.py.

4	שומר הדף (Catchword)		

העתקת התיבה הראשונה מעמוד עוקב לתחתית עמוד נוכחי.

MD

קיים: פונקציה מובנית ב-tools/catchword.py.

5 מסיר ניקוד וטעמים

סינון והסרת ניקוד/טעמים מטקסט נבחר.

MD

ועוד 1

קיים: כלי מובנה בחלונית text_editor.py.

6 קובץ כותרות (tag.hdr)

הגדרות לכותרות רצות משתנות לאורך הספר.

MD

שודרג: מוגדר ויזואלית ללא צורך בקובץ טקסטואלי, דרך dialogs.py.

7 הדפסה לייצוא (PDF Export)

מנוע המפיק קובצי PDF להדפסה.

MD

קיים: מנוהל על ידי pdf_exporter.py.

8 תצוגת שני עמודים מקבילים

הצגת עמודי ימין ושמאל יחד לבקרה.

MD

קיים: הוטמע טבעית בתצוגת ה-canvas_view.py.

9 סקריפט "עימוד" (DMD Imud)

סקריפט בתשלום ($400) המאפשר חלוקה ללא נעילת התוכנה ועריכה בזמן אמת.

MD

מובנה כברירת מחדל: ארכיטקטורת PyQt6 לא נועלת את ה-UI במהלך חישובים ברקע (QThread).

10 סקריפט "לינקס" (DMD Links)

כלי ($300) לעריכה מובנית וקלה של הערות שוליים וקישורים.

MD

מובנה: מנוהל דרך מנגנון התצוגה הויזואלי ו-link_validator.py.

11 סקריפט "מסגרות" (DMD Frames)

יצירה ועריכת מסגרות אוטומטית וידנית.

MD

מובנה: שליטה מלאה בתיבות טקסט ב-canvas_view.py בעזרת העכבר.

12 סקריפט "פייג'ס+" (Pages+)

דפדוף, מחיקה ומספור עמודים גם ברב-טקסט.

MD

מובנה: מנהל עמודים גרפי (Thumbnails) ב-main_window.py.

13 תיקון כותרות נשלפות (HDR)

פותר אוטומטית התנגשויות צפיפות בכותרות ארוכות.

MD

מובנה: אלגוריתם טיפול באילוצים ינוהל ב-justifier.py.

14 הוספת סימוני "גוף" (Guf_7)

הוספת מרווחי שורות בקליק על הדף.

MD

מובנה: הגדרות מרווח ויזואליות (Padding/Margin) ב-style_inspector.py.

15 התאמת פיסקה (Fit_7)

הצרת והרחבת רווחי מילים ואותיות לפתרון מילים בודדות.

MD

ועוד 2

מובנה: מחולק לפונקציות המיקרו-טיפוגרפיה ב-tools/auto_fitter.py.

16 מכניס קודים מתקדם (ACI)

דיאלוג להכנסת קודים ללא צורך לזכור אותם.

MD

שודרג: מתייתר לחלוטין. סגנונות מוחלים ויזואלית על בחירת טקסט ללא קודים.

17 בדיקות אוטומטיות (PLC, CPN)

חיפוש עמודים באורך חריג ובדיקת הפניות חסרות.

MD

ועוד 1

מובנה: מנגנוני בדיקה ו-Validation (התראות Pre-flight).

18 זוג השער / זוג פרק

הוספת עמוד ריק אוטומטית כדי שפרק יתחיל בעמוד זוגי אי-זוגי.

MD

מובנה: חוק פריסה הקיים ב-paginator.py (הגדרות מתחילת מקטע).

19 טורים לא מאוזנים (TL_cols)

זיהוי חוסר איזון בגובה בטקסטים מרובי טורים.

MD

מובנה: אלגוריתם ה-paginator.py יאזן אוטומטית או יתריע מראש.

20 חיפוש גלובלי (DMD GS)

חיפוש על פני קבצים רבים עם דוחות מפורטים.

MD

מובנה: תיבת חיפוש מתקדמת ב-text_editor.py.

21 אזן כותרות ארוכות (BRL)

עיצוב כותרות בצורה סימטרית על פני מספר שורות.

MD

מובנה: הגדרות Justification מיוחדות לכותרות ב-style_inspector.py.

22 חיפוש שורות צפופות/מפוזרות

איתור רווחים (Tracking) החורגים מהגדרות תקינות.

MD

מובנה: בקרת איכות טיפוגרפית (Pre-flight) ב-tools/auto_fitter.py.

23 ניהול גופנים (DMD Fonts)

תצוגה מקדימה וניהול קבוצות שימוש של פונטים.

MD

מובנה: בחירת פונטים חכמה מתוך תפריטי ה-UI של המערכת.

24 החלפת קודים וסידור טקסט (ACR, CPS, AJ)

החלפת קודים חכמה בהתניות ולפי סביבה.

MD

ועוד 2

שודרג: מוחלף בניהול סגנונות ויזואלי ברמת הפסקה (בדומה ל-InDesign).

25 סקריפט צור קובץ מפתחות (Maf)

שליפת כותרות ומספרי עמודים ליצירת תוכן עניינים.

MD

מובנה: אלגוריתם ייעודי tools/indexer.py שולף נתונים היישר מה-Data Class.

26 סקריפט "חבר" (Chaver)

מיזוג וחיבור קובצי הערות שונים לקובץ אחד ברב-טקסט.

MD

ועוד 1

מובנה: מנגנון ה-sync_manager.py יבצע Import חכם וסנכרון ממקורות שונים.

ארכיטקטורה זו מונעת את התלות של המעמד בסקריפטים חיצוניים בתשלום שעליהם מבוסס כיום המודל הכלכלי של "תג", ומעבירה את כל הלוגיקה והכלים היישר אל תוך חוויית משתמש (UI) אחת חלקה וקוהרנטית.

MD

ועוד 1

סקירה כללית

מערכת עימוד מתקדמת לספרי קודש, מבוססת PyQt6, עם ממשק משתמש מודרני ותמיכה מלאה בעברית ובכתיבה תורנית.

ארכיטקטורה ומבנה הפרויקט

```

/tag_plus

├── main.py                    # נקודת כניסה ראשית

├── /config
```

```
│   ├── __init__.py
│   ├── settings.py          # הגדרות כלליות
│   └── constants.py         # קבועים
├── /core
│   ├── __init__.py
│   ├── document_manager.py  # ניהול מסמכים
│   ├── text_engine.py       # מנוע טקסט
│   ├── multi_text_engine.py # מנוע רב-טקסט
│   ├── link_manager.py      # ניהול קישורים
│   └── page_calculator.py   # חישוב עמודים
├── /ui
│   ├── __init__.py
│   ├── main_window.py       # חלון ראשי
│   ├── ribbon_bar.py        # סרגל כלים מודרני
```

```
│   ├── editor_widget.py          # עורך טקסט
│   ├── preview_widget.py         # תצוגה מקדימה
│   └── /dialogs
│       ├── __init__.py
│       ├── page_setup_dialog.py  # הגדרות עמוד
│       ├── find_replace_dialog.py # חיפוש והחלפה
│       ├── link_dialog.py        # ניהול קישורים
│       └── multi_text_dialog.py  # הגדרות רב-טקסט
├── /features
│   ├── __init__.py
│   ├── search_replace.py         # חיפוש והחלפה מתקדם
│   ├── nikud_handler.py          # טיפול בניקוד
│   ├── teamim_handler.py         # טיפול בטעמים
│   ├── footnotes.py              # הערות שוליים
```

```
│   └── running_headers.py      # כותרות רצות
├── /import_export
│   ├── __init__.py
│   ├── word_importer.py        # ייבוא מ-Word
│   ├── word_exporter.py        # ייצוא ל-Word
│   └── pdf_exporter.py         # ייצוא ל-PDF
├── /utils
│   ├── __init__.py
│   ├── hebrew_utils.py         # כלי עזר לעברית
│   ├── font_manager.py         # ניהול גופנים
│   └── file_utils.py           # כלי קבצים
└── /resources
    ├── /icons
    └── /fonts
```

└── /templates

```

---

## רשימת קבצים לפיתוח (סדר ביצוע)

### שלב 1: תשתית בסיסית (קבצים 1-5)

1. `config/constants.py` - קבועים ותצורות בסיס

2. `config/settings.py` - ניהול הגדרות

3. `utils/hebrew_utils.py` - כלי עזר לעברית

4. `utils/font_manager.py` - ניהול גופנים
```

5. `utils/file_utils.py` - כלי קבצים

שלב 2: ליבת המערכת (קבצים 6-11)

6. `core/document_manager.py` - ניהול מסמכים

7. `core/text_engine.py` - מנוע טקסט בסיסי

8. `core/page_calculator.py` - חישובי עמוד

9. `core/link_manager.py` - ניהול קישורים

10. `core/multi_text_engine.py` - מנוע רב-טקסט

11. `features/search_replace.py` - חיפוש והחלפה

שלב 3: תכונות מתקדמות (קבצים 12-16)

12. `features/nikud_handler.py` - טיפול בניקוד

13. `features/teamim_handler.py` - טיפול בטעמים

14. `features/footnotes.py` - הערות שוליים

15. `features/running_headers.py` - כותרות רצות

16. `import_export/word_importer.py` - ייבוא מ-Word

שלב 4: ייצוא ו-UI בסיסי (קבצים 17-21)

17. `import_export/word_exporter.py` - ייצוא ל-Word

18. `import_export/pdf_exporter.py` - ייצוא ל-PDF

19. `ui/editor_widget.py` - עורך טקסט

20. `ui/preview_widget.py` - תצוגה מקדימה

21. `ui/ribbon_bar.py` - סרגל כלים

שלב 5: דיאלוגים (קבצים 22-25)

22. `ui/dialogs/page_setup_dialog.py` - הגדרות עמוד

23. `ui/dialogs/find_replace_dialog.py` - חיפוש והחלפה

24. `ui/dialogs/link_dialog.py` - ניהול קישורים

25. `ui/dialogs/multi_text_dialog.py` - רב-טקסט

שלב 6: חלון ראשי והשלמה (קבצים 26-27)

26. `ui/main_window.py` - חלון ראשי

27. `main.py` - נקודת כניסה

GEMINI: Architectural Blueprint: "Tag Plus" (PyQt6)

התוכנית שהצגת מציגה ארכיטקטורה מודולרית נכונה (Separation of Concerns). אתה ניגש לפרויקט הזה כמעין "מנוע עימוד" (Typesetting Engine) המנותק מהממשק הגרפי (UI). זו הגישה הנכונה ביותר לפיתוח תוכנה מורכבת מהסוג הזה, במיוחד לאור העובדה שחישובי עימוד (Typesetting Calculations) הם תהליכים אינטנסיביים שחייבים להתבצע ב-Background Thread כדי לא לחסום את ה-Main UI Thread.

להלן ניתוח מקצועי והמלצות לשיפור הארכיטקטורה:

Model-View-Controller (MVC) עם הפרדה חדה: המנוע (text_engine.py) לא צריך "להכיר" את ה-UI. הוא צריך לקבל טקסט ופונקציות עיצוב (Commands) ולהוציא "Display List" או "Layout Map" – מבנה נתונים שמכיר את המיקום הפיזי של כל גליף (Glyph) על העמוד. ה-preview_widget.py יצייר את המפה הזו.

הפרדת הממשק מהלוגיקה (Decoupling): מומלץ שכל מנועי הליבה (/core) יהיו פונקציונליים ככל הניתן (Pure Functions/Classes) ללא תלויות ב-PyQt6. זה יאפשר לך לבצע Unit Testing למנוע העימוד ללא צורך בהרצת GUI.

ניהול זיכרון ומבני נתונים: מאחר ומדובר בספרי קודש (טקסט רץ מורכב), השתמש במבני נתונים של Interval Trees או Doubly Linked Lists עבור ניהול פסקאות והערות שוליים. אל תנסה להחזיק את כל הטקסט במחרוזת אחת גדולה בזיכרון, אלא ב-Chunks מנוהלים.

המלצות טכניות לביצוע

1. מנוע העימוד (The Heart)

במקום קובץ יחיד, המנוע צריך להיות מורכב מ-Pipeline:

Parser: ממיר קלט (Word/Text/Tag-Codes) לאובייקטים פנימיים (DocumentObject).

Layout Engine: מחשב שבירת שורות, קרנינג, וניקוד.

Renderer: הופך את ה-Layout Map לציור על המסך (באמצעות QPainter).

אל תשתמש ב-QTextEdit המובנה. עבור עימוד תורני מדויק (כמו בתג), עליך לבצע Rendering ידני של גליפים בעזרת QPainter ושימוש ב-QFontMetrics לקבלת מידות מדויקות ברמת הפיקסל.

2. עבודה ב-Threading

הגדר את כל העבודה הכבדה של page_calculator.py ו-multi_text_engine.py כ-QRunnable או QThread שיופעל ב-QThreadPool.

סינכרון: השתמש ב-pyqtSignal כדי לעדכן את הממשק רק אחרי שהחישוב הסתיים, ולא תוך כדי, כדי למנוע Race Conditions.

3. אחסון נתונים (Persistence)

אל תסתמך רק על JSON/XML. עבור תוכנת עימוד גדולה, מומלץ להשתמש ב-SQLite כ-Backend. הוא מאפשר חיפושים מהירים (Indexing) בתוך הטקסט וניהול גרסאות (Undo/Redo) בצורה טרנזקציונית בטוחה.

רשימת צ'קליסט מורחבת (השוואה לפונקציונליות "תג")

להלן פירוט הכלים הקיימים ב"תג" והסטטוס שלהם בתכנית שלך (כולל השלמות נדרשות):

מס'	כלי/פיצ'ר	תיאור קצר	קיים בתכנית?	הערות למימוש
1	מנוע רב-טקסט שילוב טקסטים ומפרשים בדף	כן (multi_text_engine)	ליבה קריטית, דורשת אלגוריתם "קופסאות".
2	עיבוד תווים (DOS compatibility)	תמיכה בקודי תג מקוריים	לא	מומלץ להוסיף קובץ compatibility_layer.py.
3	קרנינג (Kerning)	כיוונון מרווחים בין אותיות	כן (teamim_handler?)	להעביר ל-text_engine.py.
4	חיפוש גלובלי (GS)	חיפוש בכל הפרויקט	כן (search_replace)	רצוי להוסיף אינדקס SQLite למהירות.
5	שומר דף (Catchword)	כותרת המילה הראשונה בראש עמוד	כן (running_headers)	דורש Hook בחישוב העמודים.
6	ניהול קישורים (Links)	הצלבת הערות/מראי מקומות	כן (link_manager)	לוודא תמיכה בקישורים דו-כיווניים.
7	טיפול בטבלאות	עימוד טבלאות מורכב	לא מפורט	נדרש רכיב table_engine.py.
8	ניקוד וטעמים	מיקום מדויק של סימנים	כן (nikud_handler)	דורש Rendering מותאם אישית.
9	תמיכה ב-OpenType	גופנים מתקדמים	כן (font_manager)	PyQt6 תומך בזה דרך QFont.
10	סקריפטים חיצוניים	הרצת פקודות (PRG/LIB)	לא מפורט	מומלץ להוסיף script_engine.py מבוסס Python.

הצעד הבא

התוכנית שלך בשלה מאוד. השלב הקריטי ביותר כעת הוא Proof of Concept (PoC) של ה-text_engine.py – נסה לכתוב אב טיפוס שמסוגל לרנדר עמוד טקסט בודד עם מפרש (שתי תיבות טקסט) ולבצע עליו חישוב Justification בסיסי.

האם תרצה שנתמקד בכתיבת ה-Interface (ה-Abstract Base Class) למנוע העימוד, כדי להגדיר איך הקוד שלך יתקשר עם מנוע ה-Layout העתידי?

USER: אמרת

תוכל ליצור רשימה ממוספרת במיספור אחיד לכל אורך הרשימה של פירוט כל החלקים הכלים הסקריפטים התוספות הכל הכל הכל בלי שום יוצא מן הכלל

GEMINI: Architectural Blueprint: Tag System Decomposition

כדי להחליף את מערכת "תג" (Tag) המיושנת בארכיטקטורה מודרנית (Python/PyQt6), עלינו לבצע דקונסטרוקציה של כל רכיבי הליבה והסקריפטים למערכת מודולרית. הגישה המוצעת מפרידה בין ה-Rendering Engine (מנוע הרינדור), ה- Document Object Model (DOM) (מבנה הנתונים), וה-Plugin Layer (שכבת הסקריפטים). המערכת תתבסס על ניהול מצב (State Management) מרכזי שיאפשר לממשק המשתמש להתעדכן בזמן אמת ללא צורך בהרצת סקריפטים חיצוניים כבדים.

מפת הדרכים המפורטת: רכיבי מערכת "תג" ומיפוי פונקציונלי

להלן רשימה מאוחדת וממוספרת המקיפה את כל רכיבי המערכת, הכלים והפונקציונליות הנדרשת להעתקה ושיפור:

1.0 מנוע ליבה (Core Engine)

1.1 מנוע עימוד דינמי (Dynamic Pagination Engine): חישוב גבהים, שבירת עמודים ואילוצי טקסט.

1.2 מנוע רב-טקסט (Multi-Text Engine): ניהול היררכי של מספר טקסטים מקבילים (גמרא, פירוש, רש"י, תוספות) תוך שמירה על זיקה בין מקור לפירוש.

1.3 מערכת ניהול סגנונות (Style Manager): ניהול הגדרות טיפוגרפיות (גופן, גודל, עובי, ריווח).

1.4 מנוע יישור ופיזור (Justification Engine): אלגוריתמי מילוי שורות (צפוף, רגיל, מרווח) למניעת "נהרות" בטקסט.

1.5 מערכת מונחה אובייקטים (Object Tree): תרגום מבנה ה-DOS הישן למבנה עצים (XML/JSON) הניתן לעריכה.

2.0 טיפוגרפיה ועריכה טקסטואלית

2.1 טיפול בקרנינג (Kerning Handler): התאמת מרווחים בין תווים ספציפיים (DMD Kerning).

2.2 מנוע ניקוד וטעמים (Diacritics & Cantillation Engine): תמיכה במיקום מדויק של סימני ניקוד ביחס לאותיות.

2.3 חיפוש והחלפה גלובלי (Global Search & Replace): חיפוש עם תמיכה בביטויים רגולריים (Regex) וקודים מותאמים.

2.4 טיפול באותיות גדולות/קטנות (Case Handler): המרת טקסט לטיני וניהול אותיות סופיות עבריות.

2.5 שומר דף (Catchword): מנגנון אוטומטי להעתקת המילה האחרונה של העמוד לתחתיתו.

3.0 סקריפטים ותוספות (DMD & Custom)

3.1 סקריפט "לינקס" (Links Manager): הוספה, עריכה וניהול של הערות שוליים וקישורים פנימיים.

3.2 סקריפט "מסגרות" (Frames/Am): יצירה ועריכה של תיבות טקסט ומסגרות גרפיות (ידני/אוטומטי).

3.3 סקריפט "עימוד" (Imud Tool): חלוקה לעמודים מהירה ללא נעילת המערכת (Interactive Pagination).

3.4 סקריפט "בדיקת תכונות" (Check Attributes): אימות מאפייני טקסט לפי קודים מוגדרים.

3.5 סקריפט "החל קוד ויישור" (Auto-Justify): החלת פורמט ויישור בפעולה אחת (AJ).

3.6 סקריפט "יתומים ואלמנות" (Orphans/Widows): זיהוי שורות בודדות בראש/סוף עמוד.

3.7 סקריפט "עדכון קרנינג" (Update Kerning): סנכרון בזמן אמת של טבלת הקרנינג עם התצוגה.

3.8 סקריפט "עדכון תמונות" (Update BMP): ניהול נתיבים ותקינות של אובייקטים גרפיים משורשרים.

4.0 ממשק משתמש וניהול פרויקטים

4.1 ניהול פרויקטים (Project Manager): ריכוז קבצי טקסט, הגדרות וסקריפטים לקובץ פרויקט אחד.

4.2 סרגל כלים מותאם (Custom Ribbon/Toolbar): גישה מהירה לפונקציות הנפוצות (החלפת קודים, צפייה בתוכן).

4.3 תצוגה מקדימה בזמן אמת (Real-time Preview): רינדור חזותי של העמוד ללא צורך בהדפסה ל-PDF.

4.4 ניהול גופנים (Font Management): תצוגה מקדימה וניהול ספריות גופנים (OpenType/TrueType).

5.0 קלט/פלט (I/O) ותאימות

5.1 ממיר DOCX (Word Parser): המרת קבצי Word לטקסט מקודד לתג.

5.2 מייצא PDF (PDF Exporter): הדפסת תוצרים ברמת איכות דפוס (Pre-press).

5.3 ממיר פורמטים (Format Converter): המרה בין סוגי קבצים שונים (W2M, טקסט לתג).

5.4 ממשק תסריטאי (Scripting API): יכולת הפעלת פונקציות מותאמות אישית (Python Hooks) ללא תלות בקבצי LIB סגורים.

Post-Analysis: אסטרטגיית המעבר

המעבר ל-Python/PyQt6 מחייב נטישת הגישה הסקריפטית המבוססת על קבצים (כמו ב-"תג") לטובת Event-Driven Architecture.

ביצועים: השימוש ב-PyQt6 יבטיח שה-UI לא יחסם בזמן הרצת אלגוריתמי העימוד הכבדים (שימוש ב-QThread).

מודולריות: כל סקריפט ברשימה לעיל (3.1-3.8) יהפוך ל-Module או Plugin נפרד בתוך המערכת החדשה.

תחזוקה: הארכיטקטורה תבטל את הצורך בטבלאות סקריפטים חיצוניות, שכן כל הכלים יהיו נגישים דרך ה-API הפנימי של התוכנה.

USER: אמרת

מעולה עכשיו תוכל לעשות זאת שוב בפירוט יותר בכ100 נקודות במיספור אחיד מ1 עד 100

GEMINI: כדי לבנות תוכנת עימוד תורני מודרנית (באמצעות Python ו-PyQt6) שתחליף את "תג" ותעלה עליה ביכולותיה, יש למפות את "הדנ"א" של התוכנה הקיימת ולתרגם אותו לארכיטקטורת תוכנה מודרנית.

להלן תוכנית עבודה מקיפה הכוללת 100 נקודות פיתוח ופיצ'רים נדרשים, הממוספרים רצף מ-1 עד 100.

ארכיטקטורת מנוע העימוד (1-10)

מנוע חישוב גמיש: יצירת מנוע חישוב המבוסס על אובייקטים (Object-Oriented Layout Engine).

ניהול צף: תמיכה בטקסט צף (Floating Text) המנותק ממיקום קבוע בעמוד.

דפי אב (Master Pages): הגדרת מבנה דף דינמי הכולל כותרות משתנות.

ניהול טורים: יכולת הגדרת מספר טורים משתנה בתוך אותו דף.

מניעת אלמנות ויתומים: אלגוריתם אוטומטי למניעת שורות בודדות בתחילת/סוף עמוד.

קיבוע אובייקטים: היצמדות אובייקטים (טבלאות/תמונות) למילה או לפסקה ספציפית.

מנוע שבירת שורות: אלגוריתם Justification חכם שאינו מסתמך על רווחים בלבד.

קצב העימוד (Pagination Speed): אופטימיזציה של הרינדור לעיבוד אלפי עמודים בשניות.

ניהול זיכרון: ניהול עץ ה-DOM של הטקסט בצורה יעילה (לא לטעון הכל ל-RAM).

יכולת Undo/Redo: ניהול היסטוריית שינויים מלאה לכל פעולת עימוד.

ארכיטקטורת עמוד ודפוס (11-20)

הגדרות שוליים: ניהול שוליים פנימיים/חיצוניים משתנים (Gutter).

חיתוך וסימני דפוס: הוספת סימני חיתוך, בליד (Bleed) וסימני רישום.

סגירת PDF: ייצוא ישיר ל-PDF ברזולוציה גבוהה (CMYK/RGB).

ניהול פורמטים: תמיכה בגדלים מותאמים אישית (לדוגמה: ש"ס, ספר תפילה).

ניהול שכבות (Layers): הפרדת שכבות לטקסט, הערות, ותמונות.

דפי פתיחה: הגדרות מיוחדות לשערים וכותרות.

תצוגת 'כפולות' (Spreads): עבודה ויזואלית על זוג עמודים.

ספריות גריד: שמירת תבניות גריד לשימוש חוזר.

ניתוח עמוד (Pre-flight): התרעה על שגיאות עימוד (חריגות, שורות ארוכות).

פלט דיגיטלי: ייצוא לפורמטים מונגשים (EPUB) מעבר לדפוס.

טיפול בטקסט ופארסינג (21-30)

Unicode מלא: תמיכה מלאה ב-UTF-8 וייצוג תווים בינלאומי.

מפרק קבצים (Parser): ייבוא DOCX עם שמירה על מבנה סגנונות.

טיפול בקידודים: זיהוי אוטומטי של קידודי טקסט עתיקים.

ניקוי טקסט: כלי מובנה להסרת תווים מיותרים, רווחים כפולים, וסימנים מוסתרים.

חיפוש גלובלי: חיפוש מורכב (Regex) על פני כל הפרויקט.

החלפה מותנית: החלפה המבוססת על הקשר (Context-aware search).

סנכרון דו-כיווני: עריכה ב-UI שמתעדכנת בקובץ המקור (ולהיפך).

ניהול מטא-דאטה: תיוג פסקאות (סוגיות, דיבורי מתחיל) לחיפוש עתידי.

בדיקת איות: תמיכה במילון עברית מורחב.

דיבורי מתחיל (Catchwords): מנוע אוטומטי לזיהוי ועיצוב דיבורי מתחיל.

מערכת הערות שוליים ורב-טקסט (31-40)

מנוע רב-טקסט: חישוב זרימת טקסטים מקבילים (גמרא, פירושים).

הערות שוליים דינמיות: מנוע חישוב המונע זליגת הערות לעמודים הבאים.

הפניות צולבות: מערכת קישורים פנימיים (Cross-referencing).

עיצוב הערות מותאם: שליטה ברווח בין הערות, מספרים, וסגנון.

הערות סוף פרק: מערכת ניהול הערות בסוף ספר/פרק.

סינכרון טקסטים: הצמדת מפרש לטקסט המרכזי.

ניהול הערות בגובה משתנה: יכולת הגדרת גובה מקסימלי להערות.

עיצוב מפריד הערות: קו מפריד המותאם למבנה העמוד.

הפניות מרובות: יכולת להפנות מהערה אחת למספר מקומות.

אינדקס אוטומטי: יצירת מפתח עניינים לפי מילות מפתח.

טיפוגרפיה מתקדמת (41-50)

ניהול גופנים (Fonts): ניהול קבוצות פונטים והפעלתם לפי צורך.

OpenType Features: תמיכה מלאה ב-Ligatures, Swash, וריאנטים.

Kerning אוטומטי: יבוא ושינוי טבלאות קרנינג בזמן אמת.

Tracking & Leading: שליטה מיקרוסקופית בריווח אותיות ושורות.

אותיות תלויות: טיפול אוטומטי באותיות תלויות (קיצור שורות).

יישור לשני הצדדים: אלגוריתם מתיחת שורות (Junction).

ניקוד וטעמים: מנוע ייעודי להצבת ניקוד וטעמים ללא חפיפה (Collision detection).

Drop Caps: הגדרת אות גדולה בתחילת פרק (חלון).

הדגשות: זיהוי משקלי פונט (רגיל, שמן, מוטה) בצורה אוטומטית.

סימני פיסוק: עיצוב חכם של סימני פיסוק עבריים (מרכאות, סוגריים).

סימנים מיוחדים ותווים (51-60)

לוח גליפים: ממשק בחירת תווים מיוחדים.

מנוע ביטויים: הוספת תווים מותנים (כגון: "אם סוף שורה, הוסף סימן").

סימני כותרת: עיצוב אוטומטי של כותרות רצות.

לוחות שנה/לוחות עבריים: סקריפט להוספת תאריכים.

המרת אותיות: המרה בין מספרים לאותיות (א'-ת').

סימנים מתמטיים: תמיכה בסימנים לחישובים הלכתיים.

קודים מותאמים: אפשרות להגדיר קודי טקסט לכל פעולה.

מחיקת תווים לא רצויים: ניקוי אוטומטי של "זבל" מהטקסט.

ניהול צבעים: תמיכה בצבעי Spot ו-Process.

סמלים גרפיים: הטמעת איקונים בתוך הטקסט (Inline graphics).

סגנונות ועיצוב (61-70)

סגנונות פסקה: ניהול היררכי של סגנונות.

סגנונות תו: הגדרת עיצוב למילים בודדות.

שינוי סגנון גלובלי: עדכון עיצוב בכל המסמך בלחיצה.

תבניות (Templates): שמירת מבנה שלם כקובץ תבנית.

עיצוב מותנה: החלת סגנון לפי אורך פסקה או מילה.

סגנונות מקושרים: שינוי באב מעדכן את כל הבנים.

ייבוא סגנונות: העתקת סטייל מקובץ לקובץ.

תצוגה מקדימה: צפייה בשינוי סגנון לפני החלה.

ניהול טורים: עיצוב טורים בודדים בתוך מסמך מרובה טורים.

מסגרות (Frames): יצירת מסגרות אוטומטית לפי קוד.

אוטומציה וסקריפטים (71-80)

Python API: גישה מלאה למנוע העימוד דרך פייתון.

Macro Recorder: הקלטת פעולות משתמש והפיכתן לסקריפט.

Conditional Execution: הרצת סקריפט בהתאם לנתוני טקסט.

Batch Processing: עיבוד מאות קבצים במקביל (Multi-threading).

ניהול סקריפטים: חנות/ספרייה מובנית לניהול סקריפטים חיצוניים.

Event Hooks: הפעלת פעולות לפני/אחרי סגירת קובץ.

Debugging: כלי לניקוי שגיאות בקוד/סקריפטים.

קונסולת פקודות: חלונית פקודה בדומה ל-"תג" הישן למשתמשים וותיקים.

סקריפטים מותאמים אישית: תמיכה בסקריפטים לניהול קישורים (Links).

שיתוף סקריפטים: אפשרות לייצא סקריפט כ-Plugin מוצפן.

ייצוא, ייבוא וקלט/פלט (81-90)

ייצוא ל-XML: מבנה נתונים פתוח לשימוש עתידי.

יבוא מ-Indesign: המרת קבצי IDML לפורמט הפנימי.

ייצוא ל-Word: הפיכת העימוד חזרה לקובץ וורד (לתיקונים).

ייצוא ל-Web: פורמט HTML/CSS המותאם לעימוד תורני.

שמירה אוטומטית: Auto-save חכם מבוסס טיימר.

ניהול גרסאות: שמירת גיבויים אוטומטיים.

דחיסת פרויקטים: כיווץ קבצי העימוד והתמונות לקובץ אחד.

אינטגרציה עם ענן: גיבוי ישיר לשרת ענן.

תמיכה ב-OCR: זיהוי טקסט מסריקה והפיכתו לעימוד.

ייצוא לפורמט EPUB: תמיכה ב-Reflowable text.

כלים, ממשק וחוויית משתמש (91-100)

ממשק מודרני (Dark Mode): ממשק משתמש נוח ומותאם לשעות עבודה ארוכות.

קיצורי דרך: מערכת ניהול קיצורי דרך (Hotkeys) אישית.

תצוגת רשת: הצגת רשת קווים (Grid) לעימוד מדויק.

כלי זכוכית מגדלת: זום דינמי לאזורי טקסט מורכבים.

מבנה תיקיות: ניהול פרויקט מבוסס תיקיות (טקסטים, תמונות, סקריפטים).

מולטי-פלטפורמה: תמיכה ב-Windows/Linux/Mac.

Help Center: תיעוד פנימי מובנה לכל פונקציה.

ביצועים: שילוב Rust/C++ בליבה לחישובים כבדים במידת הצורך.

API הרחבות: אפשרות למפתחים חיצוניים להוסיף כלים.

מצב "עימוד קפוא": מניעת שינויים לא רצויים לאחר סיום ההגהה.

טיפ אדריכלי: כדי ליישם את רשימה זו, מומלץ להתחיל בבניית ה-Core Engine (סעיפים 1-10) שיהיה מנותק מה-UI. לאחר מכן, ה-Parser (סעיפים 21-28) יהיה החלק המורכב ביותר – אל תשקיע ב-UI לפני שיש לך מנוע שמסוגל לעבד טקסט תורני מורכב לפורמט פנימי יציב.

USER: אמרת

מצויין ועכשיו פעם אחרונה, בפירוט מקסימלי, בכ200 נקודות במיספור אחיד, והפעם מנקודת מבט תכנית של כתיבת קוד.

GEMINI: שלום רב. מעבר לעבודה ב-Python/PyQt6 דורש תכנון הנדסי קפדני המפריד בין ה-State Management (הדאטה של העימוד) לבין ה-Rendering Engine (המנוע הטיפוגרפי). להלן תוכנית ארכיטקטונית ורשימת משימות הנדסיות מפורטת לפיתוח מערכת העימוד.

Architectural Blueprint

Model-View-Controller (MVC): הפרדה מוחלטת בין DocumentModel (מבנה העץ של הטקסט, הערות, טבלאות) לבין DocumentView (הרינדור על המסך).

Asynchronous Processing: כל חישובי העימוד (Hyphenation, Justification, Page Flow) ירוצו ב-Worker Threads באמצעות QThread כדי למנוע חסימת ה-GUI.

Plugin Architecture: החלפת הסקריפטים של "תג" ב-Python API מבוסס (Native Python Modules) שיכולים לגשת ל-API של התוכנה ללא צורך ב-External binary calls.

The Master Checklist (200 Technical Implementation Points)

I. Core Engine & Data Structure (1-40)

Define internal data representation (DOM-based tree).

Implement Node class hierarchy for text, containers, images.

Establish coordinate system (Points/Em-units).

Implement font management system (loading TTF/OTF).

Build glyph cache to optimize rendering performance.

Create memory-efficient storage for large documents.

Implement undo/redo using the Memento pattern.

.Define document metadata schema (JSON/Protobuf)

.Setup logging and telemetry for debugging layout errors

.Define styles system (Paragraph Styles, Character Styles)

.Implement global search/replace engine (Regex support)

.Create unicode normalization module

.Design text-stream buffer for O(1) character access

.Implement thread-safe access to document nodes

.Setup resource management for temporary files

.Integrate binary diffing for version control

.Design data structures for multi-text management

.Implement linkage mechanisms between footnotes and body text

.Create virtualized list containers for memory optimization

.Develop cross-platform font discovery module

.Implement RTL/LTR flow logic

.Define kerning tables storage mechanism

.Create specialized storage for "floating" objects

.Setup validation logic for data integrity (pre-serialization)

.Implement persistent file saving (incremental saves)

.Build a dependency graph for style inheritance

.Design specialized structure for multi-column layout

.Create indexing structure for rapid random access

.Implement custom memory pooling for small objects

.Setup unit tests for data structural integrity

.Define public API for external plugin interaction

.Build hook system for pre/post formatting events

.Create binary schema for high-speed I/O

.Implement cache invalidation logic for nodes

.Develop profiling tools for object memory usage

.Design locking mechanisms for multi-stream access

.Implement auto-save daemon

.Create backup/recovery protocols

.Define clear separation between core logic and UI

.Implement object serialization/deserialization logic

(41-80) II. Rendering & Typography Engine

.Implement HarfBuzz or similar text shaping library

.Build custom paragraph justification engine

.Integrate line breaking algorithms (Knuth-Plass style)

.Develop character-spacing (Tracking) engine

.Implement manual/automatic hyphenation logic

.Create grid-based alignment system

.Implement baseline alignment constraints

.Develop image and frame clipping algorithms

.Create rendering pipeline for vector-based output

.Implement print-resolution export to PDF (using Cairo/ReportLab)

.Design visual feedback for kerning adjustments

.Build font fallback system (missing glyph handling)

.Implement vertical alignment (top, bottom, middle)

.Design text-flow around arbitrary shapes

.Create specialized rendering for footnotes

.Build display logic for special characters/non-printing symbols

.Implement transparency and layer management

.Create real-time zoom/pan rendering controller

.Develop "Preview" mode with low-res asset placeholders

.Implement color space management (CMYK/RGB)

.Build pre-flight check logic (missing fonts/images)

.Design specialized rendering for table cells

.Implement master-page templates engine

.Create pagination calculation engine

.Develop dynamic text-box resizing algorithm

.Implement page-break logic (orphans/widows)

.Design custom draw calls for performance

.Build optimized repaint logic (dirty rects)

.Implement anti-aliasing controls

.Create specialized engine for Torah-layout (Rashi script/Margins)

.Design visual guides and snap-to-grid system

.Implement bookmarking for document navigation

.Build table-of-contents generation engine

.Develop cross-reference link generator

.Implement high-quality SVG rendering for imports

.Build bitmapped font cache

.Design rendering state machine

.Implement GPU-accelerated drawing options

.Create print-bleed and crop-mark generator

.Develop page numbering logic

(81-120) III. GUI Development (PyQt6)

.Setup main window with MDI or Tabbed interface

.Create dynamic toolbar architecture (context-aware)

.Implement custom widget for document canvas

.Design style inspector sidebar (dockable)

.Build property editor for selected objects

.Implement drag-and-drop system for files/text

.Create signal-slot management for UI updates

.Develop context menus (R-click actions)

.Build status bar with real-time feedback (page/char)

.Implement progress bars for long background tasks

.Create font preview dropdown component

.Design color picker tool

.Build file browser/explorer integrated in GUI

.Implement shortcut manager (custom keybindings)

.Create interactive ruler widgets

.Design search-and-replace dialog with regex

.Build user settings/preferences persistence

.Implement splash screen and initialization logic

.Create error message handling/dialogs

.Develop system tray integration

.Implement multi-language UI support

.Build theme engine (Dark/Light mode)

.Create layout presets manager

.Implement scrollbar controller with thumbnail preview

.Design object selection and bounding box logic

.Build "Page Setup" dialog (margins, bleed)

.Create print dialog with PDF settings

.Implement zoom level slider with percentage input

.Develop text-wrapping toggle in UI

.Design formatting buttons group (Bold, Italic, Superscript)

.Create paragraph formatting panel (indent, spacing)

.Implement character formatting panel (size, tracking)

.Build footnote configuration dialog

.Create TOC generation settings panel

.Design import/export options dialog

.Implement UI performance profiling (FPS monitor)

.Build specialized editor for custom scripts

.Create dockable output/log console

.Develop interactive visual guide setup

.Implement undo/redo history visualization

(121-160) IV. Scripting & Integration API

.Design Python interpreter embedding (Python.h/ctypes)

.Create "Scripting API" for document manipulation

.Develop script registry and loader

.Build sandbox environment for script execution

.Create wrapper classes for UI access (API for plugins)

.Implement script execution monitoring (timeouts/kills)

.Develop hot-reload for script development

.Create documentation generator for API

.Build boilerplate generator for custom scripts

.Implement event hooks (on_save, on_change)

.Create library for text processing (regex/replacements)

.Implement file system operations API

.Build communication channel (RPC) for plugins

.Create unit testing framework for user scripts

.Implement automatic script installer/updater

.Design permission model for scripts

.Build GUI for script management (enable/disable)

.Create logging output redirection for scripts

.Develop input validation helpers for scripts

.Implement script-based batch processing tool

(161-200) V. Legacy Conversion & Import/Export

.Develop converter for legacy "Tag" PRG files

.Build DOCX parser (using python-docx or custom xml reader)

.Implement PDF importer (conversion to editable shapes)

.Create OCR integration module (Tesseract/API)

.Build HTML/Markdown exporter

.Implement image format batch converter

.Develop mapping table for custom text encodings

.Create import assistant wizard

.Build consistency checker for imported styles

.Implement automatic cleanup of imported formatting

.Develop table structure reconstructor

.Create image path repair/relink tool

.Implement IDML (InDesign) import support

.Build XML/JSON exporter for data interchange

.Design cross-application clipboard support

.Implement placeholder handling (for missing assets)

.Build batch project processing engine

.Develop script to normalize legacy "Tag" codes

.Create report generator for conversion issues

.Implement log-based recovery for failed imports

.Setup CI/CD pipeline for automated builds

.Implement automated GUI testing (PyTest-Qt)

.Develop performance benchmarks suite

.Create documentation/Wiki generator

.Build installer creation (InnoSetup/NSIS)

.Implement error reporting/telemetry service

.Create user onboarding/tutorials system

.Develop license management/DRM logic

.Implement cloud sync/integration (optional)

.Build internal search indexer for assets

.Develop auto-updater (check remote version)

.Implement "Plugin Store" interface

.Create community resource sharing module

.Develop print-preview quality settings

.Build asset management system (embedded vs linked)

.Implement resource localization (I18N)

.Create custom keyboard layout support

.Implement high-DPI awareness logic

.Build memory leak detector

.Develop crash dump analyzer

.Define coding standards (PEP8/Linting)

.Set up automated linting in repo

.Implement pre-commit hooks for quality

.Define branching strategy (Git Flow)

.Build release packaging script

.Develop security scanning for script execution

.Implement sandbox isolation for third-party scripts

.Create system requirements checker

.Develop "Safe Mode" boot for UI debugging

.Implement hotkey conflict detection

.Build modular build system (CMake/Setuptools)

.Create dependency graph for distribution

.Implement platform-specific code paths

.Develop internal API performance testing

.Design scalable plugin architecture

.Implement session saving/restoration

.Build user feedback collection tool

.Develop documentation for internal developers

.Create sample project templates

.Final architecture review and optimization pass

Post-Code Analysis

Performance: The GUI responsiveness relies on the decoupling of the render-loop from the business logic. Using QThread and asynchronous signals is non-negotiable for large layouts. Rendering performance should prioritize dirty-rect .updates (only redraw what changed) rather than full-page refreshes

Security: By implementing a sandbox for custom scripts (using exec() within a restricted namespace or a separate process), you prevent potentially malicious scripts from accessing the underlying OS file system or network, addressing a .major risk in traditional scripting engines

Scalability: The DOM-based architecture allows for virtually unlimited page counts, provided you implement page virtualization (only loading/rendering the .visible page and its neighbors into memory)

אפיון פונקציונלי ומבני של הכלים הייחודיים בתוכנת תג לעימוד תורני

מבוא: הארכיטקטורה הטקסטואלית של תוכנת תג ומקומה בעולם ההוצאה לאור

תוכנת "תג", פרי פיתוחו של דניאל וייסמן, מהווה את התשתית הטכנולוגית המרכזית והמובילה לעימוד וסדור של ספרי קודש וספרות תורנית מורכבת.[1, 2] בעוד שתוכנות עימוד מודרניות כגון Adobe InDesign שולטות ביד רמה בעולם העיצוב הגרפי הכללי, הן נתקלות במגבלות מבניות קשות כאשר הן נדרשות להתמודד עם האתגרים הייחודיים של הספר העברי המסורתי.[2, 3] דפי גמרא, מקראות גדולות, סידורי תפילה וספרי שו"ת דורשים ארכיטקטורה דינמית המסוגלת לשלב מספר רב של טקסטים ומפרשים בעלי זיקה הדדית על גבי עמוד בודד[2].

מכיוון שהתוכנה נבנתה במקור בגרסת DOS, ממשק המשתמש שלה שונה מהותית מהממשקים הגרפיים המקובלים כיום, והיא אינה תומכת באופן מובנה בטכנולוגיות פונטים מתקדמות כגון OpenType.[1] עם זאת, חסרון זה הופך ליתרון משמעותי; הארכיטקטורה המבוססת על פקודות קוד וטקסט מאפשרת לתוכנה לבצע חישובים מתמטיים מורכבים של זרימת מלל במהירות יוצאת דופן ובדיוק מוחלט, תוך שמירה על יציבות מערכתית גם בקבצים של אלפי עמודים[1].

כדי לגשר על מגבלות הגופנים הישנות, התוכנה מציעה פתרונות ניהול פונטים ייחודיים, כגון קיצורי דרך ישירים אל מספרי זיהוי הפונטים במערכת, וכלי פנימי מובנה לבירור אילו משקלים (כגון קל, רגיל, חצי-שמן או שמן) קיימים עבור כל פונט המותקן בתוכנה.[1] בנוסף, התוכנה פוטרת את המעמד מהצורך בביצוע מניפולציות מורכבות של הזזת ניקוד וטעמים באופן ידני או באמצעות שאילתות חיפוש והחלפה מורכבות (GREP) מבוססות יוניקוד, כפי שנדרש לעיתים קרובות בתוכנות עימוד אחרות.[4] תחת זאת, תג נשענת על קבצי קרנינג (Kerning) ייעודיים המגדירים את המיקום המדויק של סימני הניקוד וטעמי המקרא ביחס לאותיות העבריות, ובכך מבטיחה מראה טיפוגרפי מושלם ואוטומטי[5].

מנגנון רב-טקסט וניהול דינמי של דפים מרובי מפרשים

הכלי המשמעותי והייחודי ביותר בתוכנת תג הוא מנגנון ה"רב-טקסט".[2] פונקציה זו מאפשרת לעמד בו-זמנית תיבות טקסט רבות בעמוד אחד, תוך יצירת סינכרון דינמי בין טקסט הבסיס המרכזי לבין המפרשים המקיפים אותו.[1, 2] האתגר המרכזי בעימוד ספרים מסוג זה הוא מניעת זליגה של מפרשים לעמודים הבאים; המערכת מחשבת את נפח הטקסט של כל מפרש ומאלצת את כולם להתכנס יחד תחת אותו עמוד של טקסט המקור אליו הם מתייחסים.[2] כל מפרש מוגדר בעל סוג הפניה ייחודי משלו, כגון שימוש באותיות, מספור שונה או גופנים ייחודיים, כדי להפריד חזותית ומבנית בין חלקי הפירוש השונים[2].

במהלך השנים האחרונות שודרג מנגנון זה באופן מהותי כדי להקל על עבודת המעמד התורני ולשפר את ביצועי התוכנה:

- **הקפאת טקסטים :(Freeze)** תכונה ייחודית זו מאפשרת למעמד להקפיא ולנעול קטעי טקסט מסוימים במהלך עימוד של "רב-טקסט".[1] פעולה זו מונעת מהתוכנה לבצע חישובים חוזרים ונשנים על דפים שכבר עומדו וסודרו, דבר המשפיע באופן מהותי על מהירות העבודה הכללית ומאפשר תצוגה מהירה וחלקה של פרויקטים רחבי היקף[1].
- **עימוד ותצוגה של שני עמודים מקבילים :**התוכנה מאפשרת להציג ולעמד שני עמודים רצופים (ימין ושמאל) בבת אחת.[1] כלי זה מאפשר למעמד לקבל מבט פנורמי על פריסת ה"לייאאוט" של הספר, ולוודא כי חלוקת המלל והאיזון הויזואלי בין הדפים נשמרים בצורה מיטבית[1].
- **לחצנים דינמיים לשינוי נפח הטקסט :**עד לעדכוני התוכנה בשנת 2024, שינוי כמות הטקסט במסגרות התאפשר אך ורק באמצעות גרירה עם העכבר, דבר שהקשה על הדיוק והאריך את זמן העבודה.[1] כיום, המערכת כוללת לחצנים ייעודיים לשינוי כמות המלל במסגרת, לצד שלושה כפתורי שליטה מובנים: "רגיל", "צפוף" ו"מרווח".[1] כפתורים אלו מבצעים מיקרו-טיפוגרפיה מיידית וממלאים את מסגרת הטקסט בצורה אופטימלית ומהירה בלחיצת כפתור[1].

פונקציות וכלים טיפוגרפיים מובנים לעימוד המקרא והתלמוד

תוכנת תג כוללת שורה של כלים טיפוגרפיים ייחודיים שנועדו לפתור בעיות עיצוב ספציפיות המאפיינות את הספר התורני, אשר בתוכנות אחרות דורשות עבודה ידנית מייגעת או תוספים חיצוניים יקרים:[2]

הסרת ניקוד וטעמי מקרא מתקדמת

התוכנה מציעה כלי ייעודי המאפשר להסיר ניקוד, או לחלופין להסיר אך ורק את טעמי המקרא, מתוך מסמך שלם או מתוך קטע טקסט נבחר.[1] כלי זה פועל באמצעות הגדרות מתקדמות המאפשרות למעמד לקבוע במדויק אילו סימנים טיפוגרפיים יימחקו ואילו יישמרו, דבר החיוני למשל בעת הפיכת טקסט חומש מנוקד ומוטעם לטקסט המיועד לתיקון קוראים או להערות שוליים פשוטות.

שומר הדף(Catchword)

בספרים עבריים קדומים נהגו להדפיס בשוליים התחתונים של הדף את המילה הראשונה שתפתח את העמוד הבא, כדי להקל על הלומד את רצף הקריאה בעת דפדוף.[1] תוכנת תג מבצעת פעולה זו בצורה אוטומטית לחלוטין; המערכת מזהה את המילה הפותחת של העמוד העוקב, ומעתיקה אותה באופן דינמי לתחתית העמוד הנוכחי ללא צורך בהתערבות ידנית של המשתמש[1].

מרכוז שורה אחרונה ופתרונות קצה למקטעים

בטיפוגרפיה התורנית נהוג שפסקאות מיושרות לשני הצדדים (Justified), אך השורה האחרונה של הפסקה אינה נצמדת לימין אלא ממורכזת בדיוק לאמצע הדף.[2] תג מאפשרת לבצע מרכוז של השורה האחרונה במקטע מבלי שהדבר יפגע באיחוד השוליים של הקטע כולו.[2] כלי זה מונע מתיחה מכוערת של מילים ומעניק לפסקה מראה אסתטי ומסורתי. בנוסף, התוכנה מאפשרת יצירת "חלון" (הזחה פנימית של הטקסט) תחת המילה הראשונה של המקטע, שהיא לרוב מוגדלת או מודגשת, לצורך עיצוב כותרות משנה פנימיות בגוף הטקסט[2, 6].

יישור תחתית העמוד וטיפול בשוליים

כלי היישור של תג מבטיח שכל עמודי הספר יסתיימו בקו תחתון אחיד ומדויק, תוך חלוקת מרווחים מבוקרת בין הפסקאות.[6] המערכת כוללת אלגוריתם מובנה לטיפול בבעיות של "יתומים" ו"אלמנות" (שורות או מילים בודדות שגלשו לעמוד הבא לבדן), ומאפשרת התאמה מדויקת של הפניות, הערות שוליים, הערות צד והערות סיום[6].

התממשקות, המרת קבצים וניהול קודים

אחד היתרונות הבולטים של תוכנת תג הוא יכולת ההתממשקות הישירה שלה עם מעבדי תמלילים נפוצים, ובעיקר Microsoft Word.[1] התוכנה מאפשרת להמיר קבצים מתג לוורד ולהפך בצורה ישירה, ללא תלות בתוכנות תיווך חיצוניות, תוך שמירה מלאה על פקודות העיצוב והקודים הפנימיים המוטמעים בטקסט[1].

תהליך העבודה המקובל מבוסס על הכנת החומר בוורד, כאשר כל סוג טקסט (כגון כותרת ראשית, כותרת משנה, ציטוט או הערה) מוגדר בגופן שונה כדי לאפשר למערכת לזהות את תפקידו המבני.[6] תג מתרגמת הבדלים אלו לקודים ופקודות עימוד באופן אוטומטי. המערכת תומכת בכלים מתקדמים להתאמת תוכן המקור, כולל:

- **יבוא וקישור הערות שוליים :**העברת קטעים מסומנים בוורד ישירות אל הערות שוליים בתג, וקישור בין טקסטים הממוקמים בקובצי וורד נפרדים ושונים[3].
- **מיקום הערות שוליים סלקטיבי :**אפשרות למקם את הערת השוליים אך ורק תחת טור אחד מתוך שני טורים של טקסט ראשי בעמוד[3].
- **קובץ הכותרות :(Kovetz Kotarot)** קובץ הגדרות מרכזי בתוכנה המכיל את כל הפונקציות העיצוביות והמבניות של הפרויקט.[7] קובץ זה מגדיר את התנהגות הכותרות, סגנונות הגופנים, רוחב העמודים והתנהגות הטקסט הדינמית[7].

מערך הסקריפטים ומנוע ההרחבה הפנימי של התוכנה

מעבר לכלים המובנים בה, כוחה של תוכנת תג טמון במנוע הסקריפטים העוצמתי שלה, המאפשר להרחיב את פונקציות המערכת ולהתאימן אישית לצרכיו של כל פרויקט.[2, 8] כדי להריץ סקריפט בתג, על קובץ הסקריפט (בסיומת PRGאו קובץ ספרייה מוגן בסיומת LIB) להיות ממוקם בתיקיית המערכת הראשית של התוכנה (בדרך כלל C:\TAG)או בספרייה הפעילה כעת, המוצגת בחלקו השמאלי התחתון של החלון.[9]

הרצת הסקריפטים מתבצעת באמצעות חלונית הפקודה הייעודית של תג, הממוקמת בתפריט העליון מימין לכפתור "G".[9] המעמד מזין פקודה מילולית פשוטה להפעלת הקובץ. לדוגמה, הפעלת סקריפט ליצירת מפתחות מתבצעת על ידי הזנת הפקודה הבאה:[9]

במידה והסקריפט מוגן בסיומת LIB, ניתן להפעיל מתוכו פונקציות ספציפיות על ידי הגדרת שם הפונקציה במרכאות.[9] לדוגמה, קריאה לפונקציית ההדבקה "Paste" מתוך קובץ ספריה מבוצעת באמצעות הנוסחה הטיפוגרפית הבאה:[9]

קהילת המעמדים והמפתחים יצרה מגוון רחב של סקריפטים המשנים מן הקצה אל הקצה את מהירות העבודה של התוכנה, והופכים משימות מורכבות לקלות ונוחות[8, 10].

אפיון והשוואה של סקריפטים וכלי עזר מובילים

כדי להציג את מגוון פתרונות ההרחבה הקיימים עבור מעמדי תג, מובאת להלן טבלה מפורטת המאפיינת את הסקריפטים המובילים בשוק, תפקידם הפונקציונלי, יוצריהם ועלותם המשוערכת:

תיאור פונקציונלי ומנגנון פעולה	עלות (בשקלים או דולרים)	מפתח / יוצר	שם הסקריפט / כלי העזר
כלי רב-עוצמה המאפשר חלוקה של הספר לעמודים מבלי לנעול את ממשק התוכנה, תוך אפשרות להפעיל סקריפטים במקביל וביצוע שינויים בקליק ישיר על הדף[10].	$400.00 [10]	DMDscripts	**עימוד(Imud)**
תוסף משלים לעריכה, הוספה וניהול של הערות שוליים ומפרשים בתוך תג בצורה נוחה ומהירה יותר ממעבדי תמלילים נפוצים[10].	$300.00 [10]	DMDscripts	**לינקס(Links)**
סקריפט המקשר בין שני קובצי וורד נפרדים על ידי שתילת קישורים והפניות מקבילים בשניהם לצורך סנכרון לפני ייבוא[11].	450ש"ח[11]	חגי פאהן	**המקושרות**
מעתיק באופן אוטומטי את הניקוד מקובץ מקור אחד ומדביק אותו בכל הציטוטים התואמים המופיעים בקובץ היעד[11].	450ש"ח[11]	חגי פאהן	**ניקוד המתחיל**
כלי ליצירה ועריכה של מסגרות בצורה ידנית או אוטומטית לפי קוד, המאפשר החלת סגנונות עיצוב מוגדרים מראש בלחיצה אחת[10].	$150.00 [10]	DMDscripts	**מסגרות (Masgerot)**
יוצר פריסת "ברך) "צורת L טיפוגרפית שבה הטקסט עוקף מסגרת אחרת (באופן אוטומטי עבור פרויקטים של רב-טקסט[11].	חינמי[11]	קובי ישורון	**ברך(Berech)**
שומר את כל נתוני העבודה הפתוחים, כולל גודל הקבצים, מיקומם, מיקום הסמן בדף ומיקום הסקריפטים הפעילים, לשחזור מיידי בקליק אחד[10].	$100.00 [10]	DMDscripts	**שמירת שלחן עבודה**

כלי עזר לדפדוף, בחירה, מחיקה ומספור של עמודים לאחר ביצוע החלוקה, התומך באופן מלא גם במערכי רב-טקסט מורכבים[10].	$100.00 [10]	DMDscripts	**פייג'ס(+Pages) +**
מאפשר למקם שורה במקביל מעל שורה אחרת בצורה נוחה (שימושי לעימוד קטעי תפילה דו-לשוניים או סידורים המשלבים כוונות מעל המילים)[11].	100ש"ח[11]	דניאל וייסמן	**אדון הכל (AdonHkol)**
גרסה משודרגת של סקריפט התאמת פסקאות, המאפשרת התאמה דינמית של נפח הפסקה ישירות מתוך דף התצוגה של רב-טקסט[10].	$100.00 [10]	DMDscripts	**פיט_7(Fit_7)**
סורק את קובץ העימוד ומאתר עמודים שחרגו מהאורך המוגדר (ארוכים או קצרים מדי), ומפיק דו"ח מפורט המאפשר ניווט מהיר לתיקון התקלות[10].	$75.00 [10]	DMDscripts	**בדיקת אורך עמודים**
מנוע חיפוש מתקדם הפועל על פני מספר רב של קבצים במקביל, ומפיק דו"ח תוצאות מפורט עם גישה מהירה למיקום המדויק של המילים שנמצאו[10].	$75.00 [10]	DMDscripts	**חיפוש גלובלי**
כלי למדידת מרחקים ובדיקת מיקומים של קטעי טקסט בדף התצוגה על ידי הדבקת קווי עזר דינמיים המשמשים כפלאג-אין אוטומטי[11, 12].	חינמי[11, 12]	דוד דבאח	**בלו ליין(BlueLine)**
מציג מקלדת וירטואלית בתחתית המסך המכילה את כל סימני הניקוד וטעמי המקרא, ומאפשר להכניסם לטקסט בלחיצה במקום שבו עומד הסמן[12].	חינמי[12]	מבוסTaam	**מקלדת עזר (Mekledet)**
מחבר מספר קובצי הערות שונים שנכתבו בנפרד עבור אותו טקסט ראשי, ומאחד אותם לקובץ הערות מסונכרן אחד[11].	200ש"ח[11]	דניאל וייסמן	**חיבור קבצים (Barot)**

סיכום ומגמות בפיתוח טכנולוגיית העימוד התורני

תוכנת תג מציגה שילוב ייחודי בין פשטות טכנולוגית ברמת הממשק לבין מורכבות אלגוריתמית יוצאת דופן בכל הנוגע לטיפול בטקסטים מרובי מפרשים.[1, 2] היכולת לנהל מערך רב-טקסט דינמי, לטפל בניקוד וטעמים בצורה מדויקת, ולשלוט בכלים טיפוגרפיים עתיקים כמו "שומר הדף" באופן אוטומטי, הופכת אותה לכלי ללא תחליף עבור מכוני המחקר וההוצאה לאור התורנית[1, 2].

העדכונים האחרונים שהוכנסו לתוכנה, כגון הקפאת טקסטים, תצוגה דו-עמודית ולחצני צפיפות דינמיים, מוכיחים כי התוכנה ממשיכה להתפתח ולהתאים את עצמה לקצב העבודה המודרני, תוך שמירה על עקרונות היסוד של הטיפוגרפיה העברית הקלאסית.[1] מערכת הסקריפטים העשירה המקיפה את התוכנה מרחיבה ללא הרף את גבולות היכולת שלה, ומאפשרת למעמדים מקצועיים להגיע לרמות גימור גבוהות ביותר, המשלבות דיוק מתמטי עם הערך האסתטי המסורתי הראוי לספרי הקודש[8, 10].

---

1. תג (תוכנה) – ויקיפדיה, https://he.wikipedia.org/wiki/%D7%AA%D7%92_(%D7%AA%D7%95%D7%9B%D7%A0%D7%94)
2. עימוד תורני לספרי קודש - טקסט רץ, https://textratz.co.il/%D7%A2%D7%99%D7%9E%D7%95%D7%93-%D7%AA%D7%95%D7%A8%D7%A0%D7%99-%D7%9C%D7%A1%D7%A4%D7%A8%D7%99-%D7%A7%D7%95%D7%93%D7%A9/
3. תוכנות העימוד – תג / אינדיזיין - עימוד ועיצוב המפואר, https://imud.co.il/%D7%9E%D7%93%D7%A8%D7%99%D7%9A-%D7%9E%D7%99%D7%95%D7%97%D7%93-%D7%9C%D7%A2%D7%95%D7%A8%D7%9B%D7%99-%D7%95%D7%9E%D7%95%D7%A6%D7%99%D7%90%D7%99-%D7%A1%D7%A4%D7%A8%D7%99%D7%9D/%D7%AA%D7%95%D7%9B%D7%A0%D7%95%D7%AA-%D7%94%D7%A2%D7%99%D7%9E%D7%95%D7%93-%D7%AA%D7%92-%D7%90%D7%99%D7%A0%D7%93%D7%99%D7%96%D7%99%D7%99%D7%9F/
4. כללי - מיקום ניקוד באינדיזיין | פרוג • קהילה עסקית - חרדית הגדולה בעולם, https://www.prog.co.il/threads/%D7%9E%D7%99%D7%A7%D7%95%D7%9D-%D7%A0%D7%99%D7%A7%D7%95%D7%93-%D7%91%D7%90%D7%99%D7%A0%D7%93%D7%99%D7%96%D7%99%D7%99%D7%9F.379568/

5. תוכנת תג - טיפול בקרנינג בתוכנת תג https://www.youtube.com/watch?v=aCtz7BdD8uY, - Debug Kerning - YouTube

6. שירותי עימוד ספרים - טקסט רץ https://textratz.co.il/%D7%9E%D7%94-%D7%90%D7%A0%D7%97%D7%A0%D7%95-%D7%A2%D7%95%D7%A9%D7%99%D7%9D/%D7%A2%D7%99%D7%9E%D7%95%D7%93/ ,

7. המדריך המלא לתוכנת תג - עימודים https://imudim.co.il/shop/%D7%94%D7%9E%D7%93%D7%A8%D7%99%D7%9A-%D7%94%D7%9E%D7%9C%D7%90-%D7%9C%D7%AA%D7%95%D7%9B%D7%A0%D7%AA-%D7%AA%D7%92/ ,

8. DMDscripts – סקריפטים לעימוד https://dmdscripts.co.il/ ,

9. סקריפטים ופונקציות ב'תג' – הוראות הפעלה - עימודים https://imudim.co.il/2019/05/15/%D7%A1%D7%A7%D7%A8%D7%99%D7%A4%D7%98%D7%99%D7%9D-%D7%95%D7%A4%D7%95%D7%A0%D7%A7%D7%A6%D7%99%D7%95%D7%AA-%D7%91%D7%AA%D7%92-%D7%94%D7%95%D7%A8%D7%90%D7%95%D7%AA-%D7%94%D7%A4%D7%A2%D7%9C%D7%94/ ,

10. מוצרים – DMDscripts - סקריפטים לעימוד https://dmdscripts.co.il/store/ ,

11. טבלת הסקריפטים לתוכנת תג - עימודים https://imudim.co.il/%D7%98%D7%91%D7%9C%D7%AA-%D7%94%D7%A1%D7%A7%D7%A8%D7%99%D7%A4%D7%98%D7%99%D7%9D-%D7%9C%D7%AA%D7%95%D7%9B%D7%A0%D7%AA-%D7%AA%D7%92/ ,

12. הורדות – DMDscripts - סקריפטים לעימוד https://dmdscripts.co.il/downloads/ ,

ארכיטקטורת מערכת העימוד "תג": מיפוי שיטתי, פונקציונלי וכלכלי של רכיבי הליבה, העדכונים ומערך התוספים והסקריפטים

מבוא: אתגרי הטיפוגרפיה התורנית והצורך בפתרונות ייעודיים

הוצאה לאור ועימוד של ספרי קודש וספרות תורנית מאופיינים באתגרים טיפוגרפיים מורכבים וייחודיים שאינם קיימים בספרות הכללית.[1] דפי גמרא, מקראות גדולות, סידורים, מחזורים וספרי הלכה מרובי פירושים דורשים ניהול סימולטני של תיבות טקסט רבות המקובעות לאותו עמוד, תוך שמירה על יחסי גומלין דינמיים וזרימה קוהרנטית ללא זליגה של הערות שוליים לעמודים הבאים.[1, 2] המורכבות הטיפוגרפית מחייבת שימוש בגופני סריף ייעודיים ובגדלים משתנים בהתאם להיררכיה הטקסטואלית המסורתית[1].

בעוד שתוכנות עימוד מודרניות כגון אדובי אינדיזיין (Adobe InDesign) מצטיינות בעיצוב גרפי ובגמישות חזותית, הן חסרות את המנגנונים המובנים לטיפול בעימוד "רב טקסט" מורכב בצורה אוטומטית ויציבה.[3, 4] מאידך, מעבדי תמלילים כגון מיקרוסופט וורד (Microsoft Word) אינם מסוגלים לספק מענה מקצועי לאתגרים גיאומטריים כגון מרכוז שורה אחרונה במקטע מבלי לפגוע באיחוד השוליים של הקטע כולו, או יצירת חלונות תחת מילה ראשונה במקטע[1].

מענה ממוקד לאתגרים אלו ניתן על ידי תוכנת "תג", שפותחה על ידי דניאל וייסמן.[2] התוכנה נבנתה במקור בגרסת DOS, ועל כן ממשק המשתמש שלה נותר מיושן יחסית ואינו תומך באופן מובנה בטכנולוגיות מתקדמות כגון OpenType [2]. למרות זאת, "תג" מהווה כיום את כלי העבודה המרכזי והבלתי מעורער לעימוד ספרי קודש[1, 2].

השוואה טכנולוגית: תוכנת "תג" מול כלי עימוד מודרניים

הניתוח הטכנולוגי והכלכלי מראה כי בחירת כלי העימוד נעשית בהתאם לצרכים הספציפיים של הפרויקט.[3] השילוב בין מספר מערכות עימוד נפוץ מאוד בענף ההוצאה לאור התורנית[3].

מיקרוסופט וורד(Word)	אדובי אינדיזיין(InDesign)	תוכנת "תג(Tag) "	מאפיין להשוואה
כתיבה, עריכת טקסטים ועיבוד תמלילים בסיסי[1, 5]	עימוד ספרות כללית, חוברות ועיצוב גרפי מתקדם[3]	עימוד ספרי קודש מורכבים ורב-טקסט דינמי[2]	**ייעוד טיפוגרפי עיקרי**

מנגנון עימוד רב-טקסט	מובנה, דינמי ומחושב מתמטית למניעת זליגת הערות[1, 2]	אינו קיים בצורה הגיונית או אוטומטית[3]	מוגבל וחסר כלים גיאומטריים מקצועיים[1]
ממשק משתמש וגרפיקה	ממשק מיושן מבוסס DOS, ללא תמיכה מובנית ב-OpenType[2]	ממשק מודרני, תמיכה מלאה בגרפיקה ובתכונות OpenType [3, 6, 7]	ממשק פשוט, תמיכה בסיסית בעיצוב ובנוסחאות[8]
מודול פיתוח והרחבות	תלות מוחלטת באקו-סיסטם של סקריפטים חיצוניים בתשלום[4, 9, 10]	תמיכה בפלאג-אינס, סקריפטים מובנים (JavaScript)ו-SDK [6, 11]	תמיכה במאקרו ותוספי עימוד ייעודיים [1]
מודל כלכלי ועלויות	רכישת רישיון חד-פעמי בליווי עלויות סקריפטים משתנות[4, 10]	דמי מנוי חודשיים קבועים) חבילת Adobe Creative Cloud) [4]	רכישה חד-פעמית או כחלק מחבילת Office 365 [8, 12]

רכיבי הליבה וההתפתחות הדינמית של המערכת

הפילוסופיה הטיפוגרפית של תוכנת "תג" נשענת על עימוד מבוסס תגיות (קודים) המוטמעות בתוך הטקסט.[13] עיקרון זה דומה לתוכנת העימוד ההיסטורית "משבצת", שפותחה על ידי משה דביר והייתה פופולרית עד סוף המאה העשרים.[13] עם זאת, בעוד שב"משבצת" הוגדרו כל התגיות בראש קובץ הטקסט עצמו והעבודה התבצעה בחלון DOS ללא עיצוב חזותי מקדים) כאשר העיצוב מוצג והודפס דרך תוכנה נלווית בשם ,[13] (DPP תוכנת "תג" פיצלה את הגדרת התגיות לקובץ סגנון נפרד והציגה תצוגה מקדימה אינטראקטיבית מובנית[2, 13].

פיתוח התוכנה לא קפא על שמריו, ובשנים האחרונות שולבו בליבת התוכנה עדכונים מהותיים המרחיבים את ביצועיה.[2] עדכוני שנת 2021 התמקדו בעיקר בייעול מהירות העבודה של מנוע הרב-טקסט ושיפור הניגשות הטיפוגרפית, בעוד שעדכוני שנת 2024 העניקו למעמדים כלים ויזואליים לשליטה ישירה בנפח הטקסט במסגרות ללא צורך בניווט מסורבל באמצעות העכבר בלבד[2].

קטלוג שיטתי ואחיד של חלקי התוכנה ותוספותיה

על מנת לספק מיפוי מלא וממצה, להלן רשימה מפורטת של כל חלקי תוכנת "תג", מאפייניה המובנים, עדכוני הליבה הרשמיים, וכן כל הסקריפטים ותוספי צד-שלישי המשלימים את סביבת העבודה, הממוספרים במיספור אחיד ורציף:

1. **מנוע עימוד הליבה של "תג (Tag Core Layout Engine)** [2] – "מערכת הפיענוח המרכזית המעבדת טקסטים בעברית ובאנגלית ומבצעת חישובים גיאומטריים לשם חלוקת עמודים ועימוד ספרים בעלי טקסט בודד או מרובה.
2. **מנוע פענוח תגיות סגנון (Style Sheet Parser)** [13] – מנגנון המפריד בין קובץ הטקסט לקובץ הסגנון החיצוני, מפענח את הקודים הטיפוגרפיים המוטמעים בטקסט ומחיל עליהם את מאפייני העיצוב המוגדרים.
3. **מנוע "רב טקסט (Rav-Text Engine)** [2] – "תשתית עימוד מתקדמת המאפשרת יצירה וניהול סימולטני של תיבות טקסט רבות בעמוד אחד, המיועדת בעיקר לעמודי גמרא, מקראות גדולות ומבנים מורכבים.
4. **מנגנון "שומר הדף (Catchword Module)** [2] – "פונקציה מובנית המחשבת ומציגה באופן אוטומטי בשולי העמוד התחתונים את התיבה הראשונה של העמוד הבא, כנהוג בטקסטים קדומים.
5. **מערכת הסרת ניקוד וטעמי מקרא (Vowel/Accent Remover)** [2] – כלי עזר מובנה המאפשר להסיר ניקוד, טעמים או שילוב שלהם ממסמך שלם או מקטע שנבחר, בהתאם להגדרות סינון מתקדמות.
6. **קובץ הכותרות ותפקודיו (Headers Configuration System - tag.hdr)** [9, 14] – קובץ הגדרות מובנה המנהל את מערך הכותרות הדינמיות והנשלפות לאורך פרקי הספר ומספק פונקציות עימוד משתנות.
7. **סרגל הכלים הטיפוגרפי (Built-in Typography Toolbar)** [9] – ממשק משתמש מובנה המרכז את פונקציות העימוד המהיר, הגדרות הגלישה והשוליים הנגישות ללחיצה.
8. **מערכת פקודות מאקרו ופלאג-אינס (Macro & Plugin Engine)** [9] – תשתית הרחבה מובנית המאפשרת למעמדים לפתח, לייבא ולהריץ פקודות מורכבות ותוספים חיצוניים המרחיבים את גבולות הליבה.
9. **מודול הדפסה וייצוא ל-PDF (PDF Export Printer)** [9] – מנוע ייצוא מובנה המפיק קובצי PDF באיכות דפוס גבוהה ישירות מתוך סביבת העבודה של התוכנה.
10. **מנגנון הרצת סקריפטים אוטומטית (Event-driven Script Runner)** [15] – תכונה שנוספה בעדכוני התוכנה ומאפשרת להפעיל סקריפט או פקודה ספציפית באופן אוטומטי בעת התרחשות אירוע מערכת מוגדר.

11. **מנגנון הקפאת טקסטים ברב-טקסט (Rav-Text Text Freezing)** – [2] עדכון מהותי משנת 2021 המאפשר "להקפיא" תיבות טקסט מעומדות במהלך חלוקת הדפים, תכונה המשפרת באופן דרמטי את מהירות העבודה של מנוע הרב-טקסט.

12. **ממשק תצוגה מתקדם לרב-טקסט (Advanced Rav-Text View)** – [2] עדכון חזותי משנת 2021 המשפר את אופן הצגת דפי העימוד מרובי הפירושים ומאפשר בקרה ויזואלית מדויקת יותר.

13. **מנגנון הגדרת קיצורי דרך בשורת התפריטים (Menu Shortcut Customizer)** – [2] עדכון משנת 2021 המאפשר הוספה והגדרה של קיצורי דרך לפקודות שונות ישירות בשורת התפריטים, כולל אפשרויות שחזור מתקדמות ותצוגת תאריך השקה.

14. **קיצור דרך אל מספרי זיהוי הפונטים (Font ID Retrieval Shortcut)** – [2] שיפור ממשק משנת 2021 המקצר את הדרך הנדרשת לשם בירור ושליפת מספרי הזיהוי הפנימיים של הגופנים המותקנים במערכת.

15. **מערכת בירור משקלי פונטים (Font Weights Inspector)** – [2] כלי בירור מובנה (עדכון 2021) המציג למעמד את רשימת המשקלים הטיפוגרפיים הזמינים עבור כל גופן ספציפי המותקן בתוכנה.

16. **מודול עימוד שני עמודים רצופים (Two-Page Sequential Layouter)** – [2] עדכון משנת 2024 המאפשר לחשב, לעמד ולחלק שני עמודים עוקבים במקביל בתוך מנוע הרב-טקסט.

17. **מנגנון תצוגת שני עמודים במקביל (Dual-Page View)** – [2] עדכון ויזואלי משנת 2024 המאפשר להציג שני עמודים מעומדים זה לצד זה על גבי מסך העבודה לשם שיפור הבקרה הטיפוגרפית.

18. **לחצני שינוי כמות טקסט במסגרת (Frame Text Volume Buttons)** – [2] עדכון ממשק משנת 2024 המאפשר להגדיל או להקטין את נפח וכמות הטקסט במסגרות הרב-טקסט באמצעות לחצני מקלדת/ממשק, במקום שימוש ידני בלעדי בעכבר שהיה נהוג בעבר ומנע עבודה רציפה.

19. **שלושה כפתורי מילוי מהיר" - Quick Fill Preset Buttons) רגיל, צפוף ומרווח** – [2] **("**תכונה שנוספה בשנת 2024 ומאפשרת לשלוט בצפיפות ובמילוי הטקסט בתוך מסגרת העימוד בלחיצת כפתור אחת מהירה.

20. **סקריפט "עימוד (DMD Imud Script)** – [14] **"**תוסף צד-שלישי המהווה פתרון מהפכני לבעיית נעילת התוכנה בעת חלוקת הדפים. תמורת מחיר של 400$, המשתמש מקבל את גרסה 4.0 (עודכנה לאחרונה ב-16 באוגוסט 2021), הכוללת את 'המדריך המלא' ושני סטים של סרטוני הדרכה

(12 סרטונים הניתנים להורדה ישירה מקובץ המדריך). הסקריפט מונע את נעילת התוכנה ומאפשר כניסה ועריכה של קובצי הטקסט ישירות בזמן החלוקה על ידי הקלקה פשוטה, לצד תיקונים מהירים בקליק בודד (פסקאות ללא קוד, צמצום שורות, שינוי גודל גופן, הפעלת/השבתת חלונות, ותיקון מילים יתומות). הוא מציג את העמוד הבא בזמן אמת, מנהל עימוד כפול ומקצר את זמן העבודה במאות אחוזים תוך חיסכון של כ-50% מהקליקים.

21. **סקריפט "לינקס" (DMD Links Script)** – [14] "תוסף משלים הנמכר בעלות של 300$, ומיועד לעריכה והוספה חוויתית וקלה של הערות שוליים וקישורים בתוך "תג" בצורה חזקה וגמישה יותר מאשר בתוכנות מעבדי תמלילים.

22. **סקריפט "מסגרות" (DMD Frames Script)** – [14] "כלי ידידותי הנמכר ב-150$, ומיועד ליצירת ועריכת מסגרות טקסט באופן ידני או אוטומטי לפי קודים מוגדרים מראש, תוך שימוש בסגנונות מסגרת הניתנים להחלה בלחיצה.

23. **סקריפט "שמירת שלחן עבודה" (DMD Save Desktop)** – [14] "כלי עזר בעלות של 100$ השומר את כל נתוני שלחן העבודה, כולל רשימת הקבצים הפתוחים, גודלם, מיקומם החזותי, מיקום הסמן בקובץ ומיקומם של סקריפטים פתוחים, ומאפשר שחזור מלא בלחיצה אחת.

24. **סקריפט "פייג'ס" (DMD Pages+)** – [14] "+כלי עזר בעלות של 100$ לניהול והצגת עמודים לאחר ביצוע החלוקה, המאפשר דפדוף, בחירה ומחיקה של עמודים, וכן ניהול מספור דפים הפעיל גם ברב-טקסט.

25. **סקריפט "תיקון כותרות נשלפות ארוכות" (DMD HDR Script)** – [14] "פלאג-אין בגרסה 2.1 (עודכן ב-26 באוגוסט 2014) הנמכר ב-100$. הסקריפט פותר בעיות צפיפות בכותרות שבהן שם הספר, שם הפרק ומספר העמוד מתנגשים, ומציע טיפול אוטומטי במהלך הדפסה, חלוקה או תצוגה. הוא כולל הגדרות מתקדמות לשינוי רווחי מילים, הצרת אותיות, שינוי גודל פונט ושבירת שורה לשורות מאוזנות.

26. **סקריפט "הכנסת סימוני גוף" (DMD Guf_7 Script)** – [14] "תוסף בגרסה 7.53 (עודכן ב-16 בנובמבר 2016) הנמכר ב-100$. הכלי מאפשר הוספה מהירה של מרווחים (גוף) בקובץ באמצעות דיאלוג ידידותי הזוכר את הערך האחרון, מחשב הוספה לקיים, מוחק סימון באמצעות הספרה "0", ומאפשר פעולות על קטעים מסומנים. בשילוב עם סקריפט "פייג'ס" הוא מציג שינויים חזותיים מיידיים ומאפשר דפדוף וקליקים מיוחדים על הדף.

27. **סקריפט "פיט_7" (DMD Fit_7 Script)** – [14] "גרסה משודרגת להתאמת פסקה הנמכרת בעלות של 100$. גרסה 7.63 (עודכנה לאחרונה ב-27 בנובמבר 2016) מציעה ממשק דיאלוג אינטראקטיבי חדש הפועל ברב-טקסט ומשתלב בקליק אחד בדף. הוא מאפשר להחיל את הפעולה על קטע

שנבחר ספציפית, מסיר את מגבלת פיצול פסקאות בסופי עמודים, ומנהל מילים בודדות ויתומות באמצעות תכונת "שאל ותקן" לפי אורך שורה בפועל. כולל שליטה ידנית במרווחי XS ו WS-באמצעות לחצני פלוס ומינוס, ומציע לחצנים מותאמים אישית בסוף הדיאלוג להחלת ערכים ספציפיים.

28. **סקריפט "מכניס קודים מתקדם (DMD ACI Script)"** – [14] תוסף בעלות של $80 המיועד לייעל את העבודה עם קודים וסגנונות, בפרט ברב-טקסט מרובה קבצים. בלחיצה אחת הסקריפט מוצא את הקוד, מכניס אותו, מוחל אוטומטית עד סוף הפסקה ומבצע יישור (בדומה לעבודה עם סגנונות באינדיזיין). הוא מציע דיאלוג ידידותי, אפשרויות לצירוף אלמנטים (קישורים, חלונות, עיטורים), הפרדת קודים, והגדרת מיקום הסמן בסיום הפעולה.

29. **סקריפט "בדיקת אורך עמודים (DMD PLC Script)"** – [14] תוסף בגרסה 1.0 (עודכן ב-28 ביוני 2024) הנמכר ב-$75. הסקריפט מבצע בדיקה אוטומטית ברקע של הקובץ ומזהה עמודים חריגים שאורכם אינו תואם להגדרות העמוד, ובכך חוסך את הבדיקה הידנית המייגעת או עצירת ההדפסה. הוא מציע שלושה סוגי הגדרות גמישות לקביעת סטייה מותרת, אפשרות להתעלמות מעמודים מסוימים (כמו עמודי שער), ומציג רשימה מפורטת עם קישורי ניווט אוטומטיים לעמוד הבעייתי, תצוגה מקדימה והפקת דוח טקסט.

30. **סקריפט "חיפוש גלובלי (DMD GS Script)"** – [14] כלי עוצמתי בעלות של $75 לביצוע חיפושים מקיפים במספר קבצים במקביל, המפיק דוח תוצאות מפורט המפרט את מספר המופעים בכל עמוד ומאפשר ניווט קל ומהיר אליהם.

31. **סקריפט "בדיקת הפניות (DMD CPN Script)"** – [14] כלי עזר משמעותי בעלות של $75 המיועד לבצע בדיקות הפניות לעמודים, תוך הצגת ממשק נוח לעבודה.

32. **סקריפט "זוג השער (DMD Zug Hashaar Script)"** – [14] כלי עזר בעלות של $65 המוסיף באופן ידני או אוטומטי עמודים ריקים בקובץ על מנת להבטיח שפרקים חדשים יתחילו תמיד בעמוד זוגי, תוך תיקון מספור העמודים בהתאמה (מתאים גם לקובצי הערות ורב-טקסט).

33. **סקריפט "חיפושים (DMD Searches Script)"** – [14] כלי עזר הנמכר ב-$65, ומציג רשימת שאילתות חיפוש עם דיאלוג ייעודי, שימושי לשמירה וניהול של חיפושים מורכבים או קבועים.

34. **סקריפט "שורה עליונה (DMD TL_imud Script)"** – [14] כלי עזר לקבצים בעלי מספר טורים בעלות של $60, המאפשר לאתר מקומות בקובץ שבהם הטורים אינם מאוזנים, ופועל הן כסקריפט רגיל והן כפלאג-אין ברקע.

35. **סקריפט "זיהוי טורים לא מאוזנים (DMD TL_cols Script)** – [14] "תוסף בגרסה 1.0 (עודכן ב-1 באוגוסט 2014) הנמכר ב-60$. הכלי מאתר חוסר איזון בגובה הטורים (שני טורים או יותר) על ידי בדיקת ראש הטור וסופו, שותל קווים מסמנים ישירות על הדף, וכותב הערה בשולי הדף המציינת את הסטייה המדויקת בנקודות. מיועד לפרויקטים מורכבים ודורש הגדרות מקדימות של גובה אות.

36. **סקריפט "חיפוש פסקאות לפי מספר שורות (DMD PBL Script)** – [14] "כלי עזר בעלות של 50$ המאפשר לחפש פסקאות במסמך על פי מספר השורות שבהן הן משתרעות (מספר מדויק, טווח, זוגי/אי-זוגי, כיוון חיפוש, סינון לפי קוד). הוא מסייע במניעת פסקאות בעלות מספר שורות אי-זוגי להתאמה אסתטית של טורים, ומאפשר החלפת קוד, החלת קוד ויישור אוטומטי מלא.

37. **סקריפט "העתק פרקים לפי קישורים (DMD CCBL Script)** – [14] "כלי בעלות של 50$ המאפשר להעתיק סימוני פרקים מקובץ אחד למשנהו בהתבסס על הקישורים הפעילים שביניהם, שימושי במיוחד להתחלת מספור הערות מחדש בכל פרק כאשר אין סימוני פרק מובנים בקובצי ההערות.

38. **סקריפט "אזן כותרות ארוכות (DMD BRL Script)** – [14] "כלי בגרסה 2.2 (עודכן ב-1 בספטמבר 2016) הנמכר ב-50$. הסקריפט מעצב כותרות ארוכות מדי וטקסטים קצרים בצורה סימטרית על ידי חלוקת המילים שווה בשווה בין השורות ושתילת שבירות שורה מאולצות (Alt + F). מיושרים לשני הצדדים (Justified), הוא מחשב ומבצע כניסה אוטומטית מהצדדים כדי למנוע רווחים גדולים ומכוערים בין המילים.

39. **סקריפט "תצוגת טקסט מתוחכם (DMD Viewit Script)** – [14] "תוסף בגרסה 2.1 (עודכן ב-18 במאי 2014) הנמכר ב-50$. הכלי מציג טקסט שנבחר בחלון תצוגה מוגדל (שימושי לעריכת קובצי קרנינג), ומציע שליטה על היקף התצוגה (שורת הבחירה, שלוש שורות, פסקה מלאה). הוא כולל מנגנון חיפוש המציג תוצאות בפונט גדול, ואפשרות לאימות החלפות על ידי סימונים זמניים ופילטורם מהתצוגה החזותית הכללית.

40. **סקריפט "חיפוש שורות מפוזרות וצפופות (DMD SM Script)** – [14] "תוסף בגרסה 1.0 (עודכן ב-9 בפברואר 2014) הנמכר ב-50$, המאתר רווחים חריגים החורגים מהגדרות רווח מינימלי ומקסימלי שהוגדרו בדיאלוג, ומציע לחצני הצרת/הרחבת רווחים מובנים לתיקון אופטימלי של השורה או הפסקה.

41. **סקריפט "פונטס (DMD Fonts Script)** – [14] "תוסף הנמכר ב-50$ ומסייע בניהול גופנים, סידורם בקבוצות שימוש (כותרות, טקסט רץ, הערות) והצגת כל הגופנים המותקנים בדף תצוגה אחד בתוך "תג". הוא מאפשר החלה מהירה של פונט בקליק מבלי לסגור את הרשימה, שינוי תכונות בזמן אמת, והשוואה חזותית סימולטנית של הטקסט הנבחר בכל הגופנים.

42. **סקריפט "סגנון פסקאות רצופות (DMD CPS Script)** – [14] "כלי עזר בעלות של $50 המאפשר להחליף קודים של פסקה בתנאי שלפניה או אחריה מופיעה פסקה המכילה קודים מוגדרים מסוימים, כולל תמיכה בהחלת קוד ויישור.

43. **סקריפט "ערוך טורים (DMD Cols Script)** – [14] "תוסף בעלות של $50 המיועד להקל על עריכת טורים בתוך הקובץ באמצעות הכנסת סימוני טורים ייעודיים.

44. **סקריפט "שאל והחלף (DMD ARP Script)** – [14] "תוסף בגרסה 2.2 (עודכן ב-16 בפברואר 2015) הנמכר בעלות של $50. הכלי משדרג את פונקציית ה"שאל והחלף" המובנית, ומאפשר להכין קובץ הגדרות מראש עם עד 10 אפשרויות החלפה שונות, שביניהן המשתמש יכול לבחור בזמן אמת באמצעות העכבר או הקשת מספר במקלדת (שימושי, למשל, לניהול מתגי קוד משתנים בתנ"ך).

45. **סקריפט "החלפת קודים מתוחכמת (DMD ACR Script)** – [14] "כלי עזר בעלות של $50 המאפשר ביצוע החלפת קודים מתקדמת המבוססת על תניות מורכבות של קודים סמוכים בתוך הפסקה.

46. **סקריפט "תפריטים (DMD Mymenus Script)** – [14] "תוסף בעלות של $40 הפותר את בעיית חוסר המקום בקובץ ה tag.ini-(המאפשר הוספה של קיצורים בשני תפריטים בלבד: קובץ וסקריפטים). הסקריפט מאפשר יצירת תפריטים חדשים ללא הגבלה, קביעת מקשי קיצור מגוונים, ואף משמש כקובץ עזר אישי מובנה לשמירת אימיילים, טלפונים או מעקבים כספיים.

47. **סקריפט "קבצים מקושרים (DMD GTL Script)** – [14] "תוסף בגרסה 1.4 (עודכן ב-11 באוגוסט 2015) הנמכר ב-$40. הוא מקצר שעות עבודה ברב-טקסט על ידי מעבר מהיר בלחיצה על כפתור "חפש קישור שבבחירה" מהטקסט הראשי ישירות למיקום ההערה המתאימה בקובץ ההערות, וחזרה מהירה בלחיצה נוספת, ללא צורך בחיפוש ידני של מספר ייחוס, אינדקס או מספר פרק.

48. **סקריפט "גובה אות הערה (DMD Govaara Script)** – [14] "כלי עזר טיפוגרפי בעלות של $40 המטפל בהרמה החזותית של הכיתוב הקטן של אות הפנייה על מנת להתאים אותה באופן מיטבי למילה שצמודה אליה.

49. **סקריפט "מיספור קבצי חבר מאחד בכל פרק (DMD NCBP Script)** – [14] "תוסף בעלות של $35 המיועד לפרויקטים של רב-טקסט, המאפשר לבצע פעולות של מיספור קישורים לפי פרק בקובץ שנוצר על ידי סקריפט "חבר."

50. **סקריפט "טיפול בקישורים באינדזיין (DMD Linked Script)** – [14] "תוסף ייעודי לתוכנת אדובי אינדיזיין בגרסה 1.01 (עודכן ב-14 בדצמבר 2016) הנמכר ב-$35. הסקריפט פותר את הקושי בביצוע שינויים גורפיים בקבצים מקושרים מרובי עמודים, ומאפשר להחיל סגנון אובייקט, העברה

לשכבה אחרת, הזזה לפי מידה או מיקום מדויק ומחיקה. כולל סינון לפי טווח עמודים, חלוקה בין עמודים זוגיים לאי-זוגיים, והחלה על כל האלמנטים בבחירה או בעמוד.

51. **סקריפט "הגדרת גובה אות (DMD TL_setup Script)"** – [14] תוסף עזר בגרסה 1.0 (עודכן ב-15 בספטמבר 2014) הנמכר ב-$35. הוא חוסך עבודה ידנית ומגדיר את גובה הפונטים בקלות, כהכנה לשימוש בסקריפטים אחרים המעבדים טקסטים בהתאם לגובה הפונט הפיזי שלהם.

52. **סקריפט "תקן רווחים קבועים (DMD RK Script)"** – [14] כלי עזר בעלות של $35 המיועד להוספה ועריכה של רווחים קבועים תוך הצגת השינויים החזותיים בזמן אמת באופן אינטראקטיבי.

53. **סקריפט "פתחן (DMD Opener Script)"** – [14] תוסף בעלות של $35 המהווה תחליף מתקדם לתיבת הדו-שיח "פתח" הבסיסית של תוכנת תג, ומעניק אפשרויות סינון, ניווט וחיפוש משופרות.

54. **סקריפט "סימניות (DMD BM Script)"** – [14] תוסף בגרסה 1.3 (עודכן ב-6 בדצמבר 2015) הנמכר ב-$35. הוא מאפשר סימון של מקומות רבים בקובץ ללא הגבלה כדי לחזור אליהם בלחיצה אחת, ללא צורך לרשום הערות ידניות. המערכת מחלקת את הסימניות לשמות וקבוצות, שומרת אותן לפי פרויקט בתוך קובץ הסגנון, ומשמשת לסימון נקודות להתייעצות עם עורכים, העתקת עיצובים, סימון בעיות עימוד או קישורים בין מקרא לפירושים.

55. **סקריפט "החל כל קוד (DMD ACC Script)"** – [14] תוסף בעלות של $35 המבצע פעולת "החל קוד" (מקוד עד קוד) ללא צורך ברישום מפורש של סוג הקוד, ומפיק רשימה עם מידע מפורט על הקודים שנבחרו.

56. **סקריפט "סינכרון קבצים (DMD Sync Script)"** – [14] כלי עזר שימושי בעלות של $25 המיועד לניווט סימולטני בשני קבצים במקביל, ומאפשר לסנכרן ביניהם על בסיס חמש אפשרויות סנכרון שונות.

57. **סקריפט "סידור והפעלת סקריפטים (DMD Runscript Script)"** – [14] כלי עזר בעלות של $25 המקל על הפעלת סקריפטים בצורה נוחה ומסודרת, תוך שימוש בממשק גישה מהיר וחדשני.

58. **סקריפט "בדיקת תכונות (DMD Checkattr Script)"** – [14] תוסף בגרסה 1.0 (עודכן ב-7 בינואר 2014) הנמכר ב-$25. הסקריפט סורק את כל הקובץ ובודק את ההרמה ,(SUP) הגוף או הפונט של כל טקסט המוגדר בקוד מסוים, ומאמת את התאמתו להגדרות קובץ הסגנון כדי למנוע שיבושים חזותיים (כגון שורות שאיבדו את הגדרת ההרמה ונוגעות זו בזו).

59. **סקריפט "החל קוד ויישור (DMD AJ Script)** – [14] "כלי עזר בעלות של $25 המחיל קוד ומיישר את הפסקה בפעולה אחת מהירה, ללא מגבלות הקשורות לסופי עמודים.

60. **סקריפט "אפשרויות בבחירת טקסט (DMD Sel Script)** – [14] "תוסף בגרסה 1.0 (עודכן ב-21 בפברואר 2014) הנמכר ב-$20. הוא מציע דרך חדשה לבחירת טקסט ללא עכבר ומיועד בעיקר לבחירת טקסט וחזרה למיקום. כולל לחצנים לבחירת עמוד או קובץ מלאים, חצים לבחירת גבולות (מהסמן ועד תחילת/סוף העמוד או הקובץ), וכפתורי "שמור מיקום" ו"קבע בחירה" הטיפוסיים לעימוד מפתחות ספרים.

61. **חוברת "המדריך המלא לתוכנת תג (Tav-Plus Complete Guide Book)** – [9] "חוברת הדרכה מקיפה המכילה למעלה מ-150 עמודי צבע מלא בפורמט גדול, המספקת הסבר מקיף לכל פקודה ופונקציה בתוכנה, כולל עימוד טקסט יחיד, הערות שוליים, עימוד בטורים, רב-טקסט, קובץ כותרות, סרגל הכלים, הדפסה ל PDF-ופקודות מאקרו.

62. **סקריפט "צור קובץ מפתחות (Built-in Index Generator)** – [15] "סקריפט מערכת נפוץ המאפשר יצירה כמעט אוטומטית של תוכן עניינים ומפתחות לספר בהסתמך על תיוג פסקאות.

63. **סקריפט "חבר (Chaver Note Combiner)** – [14] "סקריפט עזר קהילתי המשמש בעיקר בפרויקטים מורכבים של רב-טקסט לחיבור ומיזוג קבצי הערות שונים לקובץ אחיד.

64. **סקריפט "התאמת פיסקא" הבסיסי (Standard Paragraph Fitter)** – [14] סקריפט המערכת המובנה המגיע כברירת מחדל עם תוכנת תג לצורך כיווץ או הרחבה של רווחים לשם התאמת פסקאות.

ניתוח יחסי הגומלין והאקו-סיסטם הכלכלי-מקצועי של המערכת

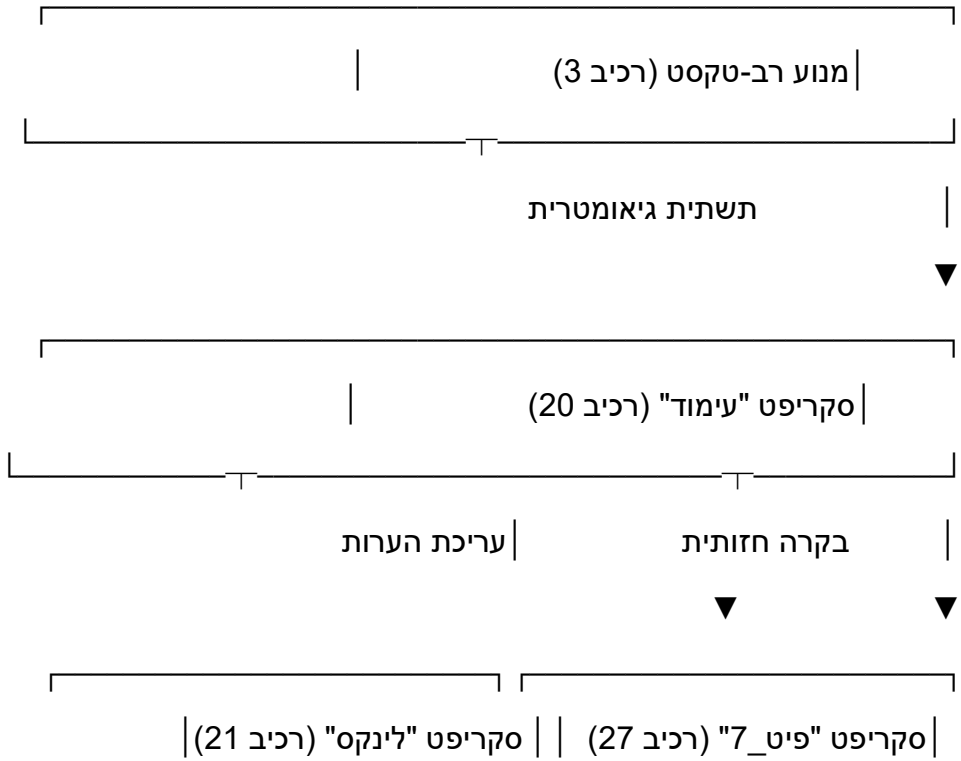
ממצאי הניתוח הטכנולוגי מעלים כי עוצמתה של מערכת "תג" אינה נובעת אך ורק מתוכנת הליבה, אלא מהסינרגיה ההדוקה ויחסי הגומלין המתקיימים בין רכיביה השונים.[2, 10, 16] מנוע ה"רב טקסט" (רכיב 3) מהווה את התשתית הגיאומטרית המורכבת של העמוד, אך הוא תלוי לחלוטין בפרודוקטיביות הנגזרת מסביבת הסקריפטים[2, 4].

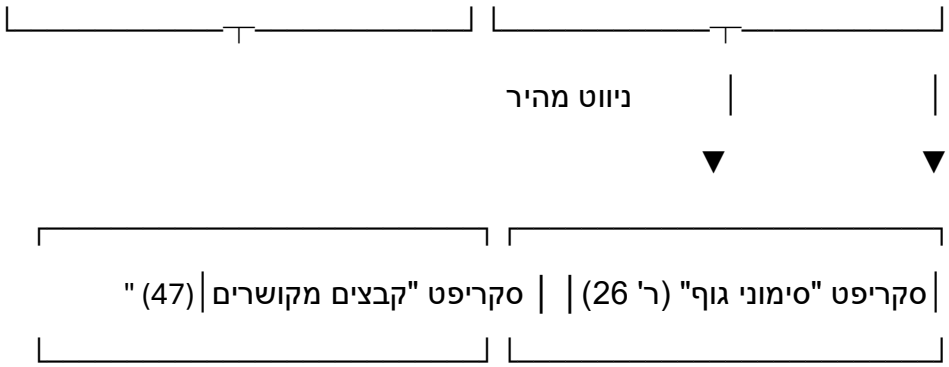
לדוגמה, במהלך העימוד הדינמי, המעמד משתמש בסקריפט "עימוד" (רכיב 20) כדי לשמור על ממשק פתוח המאפשר ביצוע תיקונים בזמן אמת.[14] במידה והמעמד מזהה עמוד ארוך מדי, הוא מפעיל מתוך סקריפט העימוד את סקריפט "פיט_7" (רכיב 27), אשר מחשב ומבצע כיווץ אופטימלי של רווחי

XS וWS-ישירות על גבי הדף, ומבצע אינטגרציה עם סקריפט "הכנסת סימוני גוף" (רכיב 26) על מנת להצר או להרחיב מרווחים בין פסקאות בצורה חזותית ומיידית[14].

השילובים הטיפוגרפיים מתבטאים גם במערכות הניווט והבקרה של הספר. סקריפט "קבצים מקושרים" (רכיב 47) מאפשר מעבר חזותי מהיר בין המקרא להערות, ומגיע לכדי מימוש מלא בתוך סקריפט "לינקס" (רכיב 21), המשמש כסביבת עבודה שלמה לניהול הקישורים[14].

בספרים מרובי טורים, המעמד נעזר בסקריפט "זיהוי טורים לא מאוזנים" (רכיב 35), הדורש הכנה מוקדמת של הגדרות גובה אות המבוצעת באמצעות סקריפט "הגדרת גובה אות" (רכיב 51).[14] קשרים פונקציונליים אלו מוצגים בתרשים הבא:





מבחינה כלכלית ומקצועית, קהילת המעמדים התורנית פיתחה מודל השקעה מבוסס תוספים.[4] בשונה מאדובי אינדיזיין הפועלת במודל של דמי מנוי חודשיים [4], תוכנת "תג" נרכשת ברישיון חד-פעמי מדניאל וייסמן, אך ביצוע עימוד מקצועי מחייב רכישת סקריפטים מותאמים אישית שמחיריהם נעים בין עשרות למאות דולרים לפריט.[2, 4] השקעה זו נחשבת לכדאית ביותר עבור המעמדים, שכן היא מייעלת את תהליך הייצור, מקצרת מאות שעות עבודה ידניות ומאפשרת להגיע לתוצאות טיפוגרפיות מדויקות שאינן ניתנות להשגה באף כלי אחר[14, 16].

תהליכי המרת קבצים ואינטגרציה בסביבה רב-תוכנתית

סביבת העימוד התורנית דורשת תהליכי אינטגרציה ומעבר קבצים מורכבים בין מערכות שונות.[2, 5] קובצי המקור נכתבים לרוב במיקרוסופט וורד, ועוברים המרה ל"תג" תוך שמירה מלאה על התגיות והקודים המובנים[2].

במקרים שבהם נדרש לייצא טקסט מאינדיזיין חזרה לוורד לצרכי הגהה ועריכה, קיימים מספר מנגנוני מעבר:

- **מנגנון ייצוא באמצעות אדובי אקרובט פרו (Acrobat Pro Loop):** המעמד מייצא את קובץ האינדיזיין (INDD) לפורמט PDF באיכות הדפסה גבוהה) בדרך כלל בהגדרות PDF/X-4 השומרות על מרחב הצבע המקורי של התמונות [17, 18, 19].(לאחר מכן, הקובץ נפתח באקרובט פרו ומיוצא לפורמט Word Document (.docx).[17, 18, 19] המרה זו מותירה את העימוד והתוכן פתוחים לעריכה, אך אינה מומלצת לשימוש מקצועי ישיר ללא הגהה מוקדמת, עקב שגיאות פריסה נפוצות[19].

- **סקריפט Export All Stories מובנה:** אינדיזיין מציעה סקריפט מובנה המעתיק את כל תיבות הטקסט (Stories) לקובצי טקסט נפרדים בפורמט Text Only או RTF השומר על עיצוב בסיסי.[11] לשם איחוד כל הסיפורים לקובץ וורד בודד, המעמדים משתמשים בתוסף חיצוני כגון Text Exporter.[11]
- **תוספי המרה ישירים של Markzware:** פתרונות כגון PDF2DTP או שירותי הענן MarkzPortal מאפשרים להמיר קובצי PDF (שנוצרו מוורד) ישירות לפורמציות אינדיזיין לעריכה מלאה ללא אובדן איכות, תוך שמירה על טבלאות, גופנים ורכיבי SmartArt.[20, 21]

מעברים אלו מלווים לעיתים קרובות בשגיאות עיצוביות, ועל כן דורשים הכנות קפדניות.[19, 22] בעת מעבר מאינדיזיין לוורד, המעמד נדרש להמיר את כל התמונות והאיורים לפורמט JPEG באמצעות מעבדי תמונה כמו פוטושופ כדי למנוע שיבושים, להסיר מעברי מקטע מיותרים, ולהתאים את כותרות Heading 1 ו Heading 2-של וורד להגדרות סגנונות הפסקאות של אינדיזיין על מנת לשמור על היררכיה נקייה[22].

סיכום והמלצות מקצועיות

ממצאי המחקר מצביעים על כך שתוכנת "תג" נותרה עמוד התווך המרכזי של ההוצאה לאור התורנית, בעיקר בזכות מנוע הרב-טקסט הדינמי שלה שאין לו תחליף מקצועי בשוק הכללי.[2, 3] עם זאת, שימוש יעיל בתוכנה מחייב הצטיידות במערך תוספים המותאם לאופי העבודה[4, 10].

למעמדים מקצועיים מומלץ להשקיע בתוספי הליבה הניהוליים – כגון סקריפט "עימוד" (רכיב 20) וסקריפט "פיט_7" (רכיב 27) המקצרים משמעותית את זמני העימוד ומונעים את נעילת התוכנה[14].

שילוב נכון בין יכולות העימוד הגיאומטריות הבלתי מוגבלות של "תג" לבין יכולות הייצוא והעיצוב של אינדיזיין, תוך שמירה על תהליכי המרת קבצים מבוקרים, מהווה את הדרך הטובה ביותר להפקת ספרי קודש העומדים בסטנדרטים הטיפוגרפיים ההיסטוריים והמודרניים כאחד[3, 19, 20].

---

1. עימוד תורני לספרי קודש - טקסט רץ , https://textratz.co.il/%D7%A2%D7%99%D7%9E%D7%95%D7%93-%D7%AA%D7%95%D7%A8%D7%A0%D7%99-%D7%9C%D7%A1%D7%A4%D7%A8%D7%99-%D7%A7%D7%95%D7%93%D7%A9/
2. תג (תוכנה) - ויקיפדיה , https://he.wikipedia.org/wiki/%D7%AA%D7%92_(%D7%AA%D7%95%D7%9B%D7%A0%D7%94)

3. תוכנות העימוד – תג / אינדיזיין - עימוד ועיצוב המפואר- https://imud.co.il/%D7%9E%D7%93%D7%A8%D7%99%D7%9A-%D7%9E%D7%99%D7%95%D7%97%D7%93-%D7%9C%D7%A2%D7%95%D7%A8%D7%9B%D7%99-%D7%95%D7%9E%D7%95%D7%A6%D7%99%D7%90%D7%99-%D7%A1%D7%A4%D7%A8%D7%99%D7%9D/%D7%AA%D7%95%D7%9B%D7%A0%D7%95%D7%AA-%D7%94%D7%A2%D7%99%D7%9E%D7%95%D7%93-%D7%AA%D7%92-%D7%90%D7%99%D7%A0%D7%93%D7%99%D7%96%D7%99%D7%99%D7%9F/ ,

4. הלימוד תוכנת תג והדרך להשתמש!!!! | פרוג • קהילה עסקית - חרדית הגדולה בעולם- https://www.prog.co.il/threads/%D7%94%D7%9C%D7%99%D7%9E%D7%95%D7%93-%D7%AA%D7%95%D7%9B%D7%A0%D7%AA-%D7%AA%D7%92-%D7%95%D7%94%D7%93%D7%A8%D7%9A-%D7%9C%D7%94%D7%A9%D7%AA%D7%9E%D7%A9.719644/ ,

5. חנות הסקריפטים - עימודים https://imudim.co.il/shop/ ,

6. Convert PDFs to InDesign documents - Adobe Help Center, https://helpx.adobe.com/indesign/desktop/create-and-organize-pages/import-and-convert-file-to-indesign/convert-pdfs-to-indesign-documents.html

7. שימוש בגופנים ב-InDesign - Adobe Help Center, https://helpx.adobe.com/il_he/indesign/using/using-fonts.html

8. עיצוב טקסט או מספרים ככתב עילי או ככתב תחתי - תמיכה של Microsoft, https://support.microsoft.com/he-il/office/%D7%A2%D7%99%D7%A6%D7%95%D7%91-%D7%98%D7%A7%D7%A1%D7%98-%D7%90%D7%95-%D7%9E%D7%A1%D7%A4%D7%A8%D7%99%D7%9D-%D7%9B%D7%9B%D7%AA%D7%91-%D7%A2%D7%99%D7%9C%D7%99-%D7%90%D7%95-%D7%9B%D7%9B%D7%AA%D7%91-%D7%AA%D7%97%D7%AA%D7%99-3649411b-adf4-483e-b0e8-7b844605da74

9. המדריך המלא לתוכנת תג - עימודים- https://imudim.co.il/shop/%D7%94%D7%9E%D7%93%D7%A8%D7%99%D7%9A-%D7%94%D7%9E%D7%9C%D7%90-%D7%9C%D7%AA%D7%95%D7%9B%D7%A0%D7%AA-%D7%AA%D7%92/ ,

10. הכל למעמד - עימודים, https://imudim.co.il/%D7%94%D7%9B%D7%9C-%D7%9C%D7%9E%D7%A2%D7%9E%D7%93/ ,
11. How to export text from InDesign | Redokun Blog, https://redokun.com/blog/export-text-from-indesign
12. המרת טופס או בוחן Word או PDF למסמך- Microsoft Forms, https://support.microsoft.com/he-il/forms/convert-a-word-or-pdf-form-or-quiz-to-microsoft-forms
13. משבצת (תוכנה) - ויקיפדיה, https://he.wikipedia.org/wiki/%D7%9E%D7%A9%D7%91%D7%A6%D7%AA_(%D7%AA%D7%95%D7%9B%D7%A0%D7%94) ,
14. מוצרים – DMDscripts, https://dmdscripts.co.il/store/
15. עימודים – עימוד ספרים והפקות דפוס, https://imudim.co.il/ ,
16. DMDscripts – סקריפטים לעימוד, https://dmdscripts.co.il/ ,
17. Convert InDesign to Word FAST Without Losing Formatting - YouTube, https://www.youtube.com/shorts/oMYIV5U23Us
18. Quickly Convert an InDesign File to Microsoft Word & Keep the Layout Design & Styles, https://www.youtube.com/watch?v=aJzyKvgBRB4
19. InDesign to Word: 4 Minutes tutorial | Redokun Blog, https://redokun.com/blog/indesign-to-word
20. How to Convert Word to InDesign for Prepress Graphic Design: PDF2DTP - Markzware, https://markzware.com/pdf2dtp/how-to-convert-word-to-indesign-prepress-graphic-design-pdf2dtp/
21. How to Convert Adobe InDesign Files Without Adobe (2025 Guide) - Markzware, https://markzware.com/workflow-tips/blog-convert-indesign-files-without-adobe/
22. Convert Indesign to word - the right way - YouTube, https://www.youtube.com/watch?v=jwfbBkf5W3I